Johannes Kepler University Linz
[Name of the institute]

Linearizing Llama-3.2-1B with Lizard

A Reproduction Study and a Search for the Gap to the Paper

Master's Thesis

to get the academic degree in [Name of the study program]

Author
Mahdi Khashan

Supervisor
[Name of the supervisor]

October 2026

Draft. This text is a first full draft. It comes from the project notes in the repository `mahdikhashan/lolcats` (folder `docs`). The author must read and check every chapter before the submission. Appendix G describes the use of AI tools.

Abstract

Softmax attention needs more time and memory when the text gets longer. Linearization changes a trained language model so that it uses linear attention with a fixed-size state. Lizard is a linearization method. It replaces each softmax attention layer with two parts. The first part is a gated linear attention over all earlier tokens. The second part is a softmax attention over a short window of recent tokens, with four sink logits. Lizard trains these parts in two stages. Stage 1 copies the output of each teacher attention layer. Stage 2 trains low-rank adapters with the next-token loss. The Lizard paper reports results near the teacher for Llama-3.2-1B.

This thesis reproduces Lizard on Llama-3.2-1B in the open LoLCATs code base. The first reproduction did not reach the paper. It got 23.3 on MMLU, 68.0 on PIQA and 54.8 on ARC-Easy, against 29.8, 74.8 and 65.6 in the paper. Tests show that the code computes the formulas of the paper. The search for the cause tested the training recipe, the numerical precision, the readings of the paper and the design of the attention. A lower stage 1 loss did not give a higher accuracy. Four interpretability methods pointed to the main cause. The window branch has no rotary position embedding (RoPE), so it cannot reproduce the local attention of the teacher.

A window with RoPE gives 73.1 on PIQA and 63.8 on ARC-Easy after stage 2. These scores are 98.2% and 97.7% of the teacher in the same evaluation harness. On all 14,042 MMLU questions, the model gets 27.2, against 31.7 for the teacher. Thus the model is above chance, but it keeps only one third of the part of the teacher score above chance. The thesis reports this model as an extension of Lizard, not as a reproduction. It also lists the open questions about MMLU, the efficiency at long inputs and the effect of the random seed.

Keywords: linear attention, linearization, distillation, Lizard, LoLCATs, rotary position embedding, reproduction study, interpretability

Declaration of the Use of AI Tools

The author used AI tools in this work. This page gives a short statement. Appendix G gives the details.

- **Code and project notes.** Claude Code wrote most code changes and project notes as pull requests. The author gave each task, reviewed the pull requests and merged them. The repositories keep the full history.
- **Answers from ChatGPT.** Some project notes keep answers from ChatGPT without changes. This thesis uses them only as ideas. It does not use them as sources of facts.
- **This text.** Claude Code wrote the first draft of this thesis from the project notes. The author must check every statement, number and reference before the submission.

Linz, October 2026

Mahdi Khashan

Contents

Abstract i

Declaration of the Use of AI Tools ii

Abbreviations and Names viii

1 Introduction 1
- 1.1 Softmax attention is expensive for long texts . . . 1
- 1.2 Linearization: a trained model gets linear attention . . . 1
- 1.3 The Lizard method and its claims . . . 1
- 1.4 Research questions . . . 2
- 1.5 Contributions . . . 2
- 1.6 Structure of the thesis . . . 2

2 Background 4
- 2.1 The transformer and causal softmax attention . . . 4
- 2.2 Rotary position embedding . . . 4
- 2.3 The cost of softmax attention . . . 5
- 2.4 Linear attention and feature maps . . . 5
- 2.5 Hedgehog feature maps . . . 5
- 2.6 Gated linear attention . . . 6
- 2.7 Sliding window attention and attention sinks . . . 6
- 2.8 Knowledge distillation and attention transfer . . . 6
- 2.9 Low-rank adaptation . . . 7
- 2.10 How the harness scores multiple-choice tasks . . . 7
- 2.11 The benchmarks . . . 8

3 Related Work 9
- 3.1 Linear attention and recurrent models . . . 9
- 3.2 Linearization methods: LoLCATs, Liger and Lizard . . . 9
- 3.3 Linearized models at the 1B scale . . . 9
- 3.4 Hybrid models and retrieval heads . . . 11
- 3.5 Interpretability of distilled attention . . . 11

4 Lizard and Its Implementation 12
- 4.1 Lizard attention . . . 12
- 4.2 The two training stages . . . 12
- 4.3 The training algorithm in log space . . . 13
- 4.4 The reference implementation . . . 14
- 4.5 The port to LoLCATs . . . 15
- 4.6 Tests of the attention layer . . . 15
- 4.7 The paper against the code . . . 16
- 4.8 Lizard attention v2: one option for each reading . . . 17
- 4.9 Proofs and float64 tests . . . 17
- 4.10 Lizard, LoLCATs, GLA and Liger in formulas . . . 17

4.11 Numerical stability: the NaN crash . . . 18

5 Experimental Setup **19**
5.1 The teacher and the data . . . 19
5.2 The training recipes . . . 19
5.3 Infrastructure . . . 19
5.4 Compute and cost . . . 20
5.5 The evaluation . . . 21
5.6 Statistics . . . 22
5.7 The teacher in this harness . . . 22
5.8 The models of this thesis . . . 22

6 Results of the Reproduction **24**
6.1 Runs 0, 1 and 2 . . . 24
6.2 The scores against the paper and the teacher . . . 24
6.3 The model selects “A” on MMLU . . . 25
6.4 The model uses both branches . . . 26
6.5 The gap starts in stage 1 . . . 26
6.6 The temperature does not change the scores . . . 27
6.7 Summary of the reproduction . . . 27

7 The Search for the Cause of the Gap **28**
7.1 The possible causes . . . 28
7.2 Precision: bf16 can discard small updates . . . 28
7.3 The recipe experiments . . . 29
7.4 Stage 2 with the recipe of the paper . . . 30
7.5 A lower loss does not give a higher accuracy . . . 30
7.6 A limit for each head . . . 31
7.7 Design options of v2: C1, `joint` and `hybrid` . . . 32
7.8 The LoLCATs attention as a control . . . 33
7.9 Summary . . . 33

8 Analysis with XAI Methods **34**
8.1 Layer-wise MSE . . . 34
8.2 Sample attention weights . . . 35
8.3 MSE by token position . . . 36
8.4 Branch decomposition . . . 37
8.5 Why short prompts need a good window . . . 38
8.6 What the XAI results show together . . . 38

9 RoPE in the Window Branch **39**
9.1 Why RoPE in the window, and the predictions . . . 39
9.2 What RoPE in the window means for the linearization . . . 39
9.3 Results after stage 1 . . . 41
9.4 Results after stage 2 . . . 42
9.5 The shared denominator does not help . . . 43
9.6 Table 9 of the paper . . . 44
9.7 MMLU on all questions . . . 45
9.8 Letters and ties on MMLU . . . 46

10 Discussion **48**
10.1 Answers to the research questions 48
10.2 Why PIQA and ARC-Easy recover, and MMLU does not 48
10.3 Differences from the paper that stay 49
10.4 Limits of this study 49
10.5 Lessons for reproduction work 49

11 Conclusion and Future Work **51**
11.1 Summary 51
11.2 Future work 51

A Definitions **52**

B Formulas and Proof Sketches **54**
B.1 The parallel form equals the recurrent form (P1) 54
B.2 The total weight of a row and the window share (P2, P8) 54
B.3 Four equal sinks act as one sink (P7) 54
B.4 The written sink term can break the denominator (P4) 55
B.5 The log-space form and the softmax map (P3) 55
B.6 The shared denominator (`hybrid`) 55

C Configurations of the Runs **56**

D All Runs and Scores **58**

E MMLU for Each Subject **59**

F How to Reproduce the Results **62**
F.1 Training on Hugging Face Jobs 62
F.2 Evaluation on a GPU machine with conda 62
F.3 XAI scripts 63
F.4 This document 63

G Use of AI Tools **64**
G.1 Claude Code 64
G.2 ChatGPT 64
G.3 This thesis 64

List of Figures

2.1 Keys that each type of attention uses 7

4.1 One Lizard attention layer 13
4.2 The two training stages of Lizard 14

5.1 The path from a code change to a result 20

6.1 The first Lizard model against the teacher and the paper 25

7.1 Timeline of the project 28
7.2 PIQA and ARC-Easy after stage 1 and after stage 2 31
7.3 Stage 1 loss against the accuracy after stage 1 31

8.1 Layer-wise MSE of all Lizard stage 1 runs and the LoLCATs control . . 34
8.2 Attention weights of head 0 in five layers 36

9.1 Weight that the gate keeps after n tokens 40
9.2 Layer-wise MSE of `window_rope` against Run 1 42
9.3 Branch decomposition at positions 0–127 42
9.4 Relative MSE by layer and position bucket 43
9.5 PIQA and ARC-Easy as a share of the teacher 44
9.6 MMLU by subject group 46
9.7 MMLU for each subject 46
9.8 Selected letters on all MMLU questions 46

List of Tables

2.1 The benchmarks of this thesis 8

3.1 Four attention methods side by side 10
3.2 Reported results of linearized Llama-3.2-1B models 10

4.1 The Lizard parameters of one layer 12
4.2 The reference pipeline against the LoLCATs pipeline 15
4.3 Open points of the paper 16
4.4 The options of Lizard attention v2 17
4.5 Main claims of the proof audits 18

5.1 Data of both training stages 19
5.2 The LoLCATs recipe against the recipe of the paper 20
5.3 Measured time and cost 21
5.4 Facts about the harness that affect the results 21
5.5 The teacher in this harness and in the paper 22
5.6 The main models of this thesis 23

6.1 Scores of the first Lizard model 24
6.2 Accuracy of the first Lizard model against fixed answers 25
6.3 PIQA with one branch removed at inference 26
6.4 The first Lizard model after each stage 26
6.5 The MMLU subset at four temperatures 27

7.1 Ranking of the possible causes after the first reproduction 29
7.2 Stage 1 runs with different recipes 30
7.3 Stage 1 runs with options of Lizard v2 32
7.4 The LoLCATs control against Lizard after stage 1 33

8.1 Results of the branch decomposition 37
8.2 Prompt lengths of PIQA and ARC-Easy 38

9.1 Predictions before the `window_rope` run and the results 39
9.2 Where each method uses RoPE 41
9.3 `window_rope` after stage 1 41
9.4 `window_rope` after stage 2 43
9.5 `window_rope` with and without `hybrid` 44
9.6 `window_rope` against Table 9 of the paper 45
9.7 MMLU on all questions 45

C.1 Config files of the runs 56

D.1 All runs and scores 58

E.1 MMLU accuracy for each subject 59

Abbreviations and Names

Short form	Meaning
ARC	AI2 Reasoning Challenge, a benchmark of science questions (Easy set and Challenge set)
AWA	Anchor window attention. The paper's name for the window branch.
bf16	bfloat16, a 16-bit floating-point format
GLA	Gated linear attention
G&A	Gather-and-aggregate heads
KL	Kullback–Leibler divergence
LoRA	Low-rank adaptation
MMLU	Massive Multitask Language Understanding, a benchmark of 57 subjects
MSE	Mean squared error
PIQA	Physical Interaction Question Answering, a benchmark
RoPE	Rotary position embedding
SE	Standard error
SWA	Sliding window attention
TV	Total variation distance
XAI	Explainable AI. Here: methods that show how the model works inside.
teacher	The unchanged model `meta-llama/Llama-3.2-1B`
first Lizard model	The model of the first reproduction: stage 1 of Run 1 and stage 2 of Run 2
`window_rope`	Lizard with RoPE in the window branch. It is the best model of this work.
gated branch	The gated linear attention part of Lizard
window branch	The softmax attention over the last 128 tokens, with 4 sink logits
Lizard parameters	The parameters that Lizard adds to each layer: the feature maps, the gate, the sink logits and α
harness	The LM Evaluation Harness at commit `b281b09`
MMLU subset	285 MMLU questions, 5 from each subject
point	One percentage point

CHAPTER 1

Introduction

1.1 Softmax attention is expensive for long texts

Large language models use the transformer architecture [27]. In each layer, softmax attention compares the current token with all earlier tokens. Thus the time for a full sequence grows with the square of its length. During text generation, the model also keeps the keys and the values of all earlier tokens in a cache. This cache grows with the length of the text.

For Llama-3.2-1B, the cache needs approximately 32 KiB for each token. This estimate uses 16 layers, 8 key-value heads, a head dimension of 64 and 2 bytes for each value. A text of 128,000 tokens then needs approximately 4 GiB of cache for one sequence. Long inputs, many users at the same time and small devices all make this cost a problem.

1.2 Linearization: a trained model gets linear attention

Linear attention replaces the softmax with a product of feature maps [14]. Then the model can keep a state of fixed size in place of the cache, and the time grows only linearly with the length. A model with linear attention can be trained from the start. But this needs a full pretraining run, which is very expensive.

Linearization is a cheaper path. It starts from a trained softmax model and keeps most of its weights. Only the attention layers change. A short training then teaches the new attention to copy the old one. LoLCATs [33], Liger [18] and Lizard [20] are three methods of this type.

1.3 The Lizard method and its claims

Lizard replaces each attention layer with two branches. A *gated branch* uses gated linear attention (GLA) [30] with Hedgehog feature maps [32]. It covers all earlier tokens with a state of fixed size. A *window branch* uses softmax attention over the last 128 tokens. It adds four learnable sink logits to its denominator. The output is the gated branch plus a learnable weight α times the window branch. Training has two stages. Stage 1 trains only the new Lizard parameters, so that each layer copies the output of the teacher layer. Stage 2 trains low-rank adapters (LoRA) [13] with the next-token loss.

For Llama-3.2-1B, Table 9 of the paper reports 74.8 on PIQA, 65.6 on ARC-Easy and 29.8 on MMLU. The teacher has 74.1, 65.4 and 31.0. Thus the paper describes Lizard as almost equal to the teacher at this size.

1.4 Research questions

This thesis follows the order of a reproduction study: the claim, the reproduction, the differences, the search for the cause and the conclusion. Four research questions give this order.

RQ1 Can the Lizard results of the paper for Llama-3.2-1B be reproduced?

RQ2 Which differences in the implementation, the training recipe and the evaluation explain the gap to the paper?

RQ3 Which changes to the Lizard attention close the gap?

RQ4 Do measurements inside the model explain the scores? These measurements are the layer errors, the attention weights, the gates and the shares of the two branches.

1.5 Contributions

1. **An open implementation with tests.** Lizard attention runs inside the LoLCATs code base. Its formulas match a slow reference implementation to a relative error below 10^{-10} in float64 (Chapter 4).
2. **A documented reproduction.** The work records every training run, its recipe, its cost and its evaluation. A full run of both stages costs approximately $19 on one H200 GPU (Chapters 5 and 6).
3. **A search for the cause of the gap.** Eight stage 1 variants test the recipe, the precision and the readings of the paper. A control run with the LoLCATs attention shows that the cause is in the attention layer (Chapter 7).
4. **Interpretability tools for distillation.** Four scripts measure the error of each layer, the attention weights, the error at each token position and the share of each branch (Chapter 8).
5. **RoPE in the window branch.** This change gives 98.2% of the teacher on PIQA and 97.7% on ARC-Easy. On MMLU, it gives 27.2 against 31.7 for the teacher (Chapter 9).

1.6 Structure of the thesis

Chapter 2 explains attention, linear attention, gates, windows, distillation and the evaluation. Chapter 3 describes related methods. Chapter 4 gives the Lizard method,

its implementation and its tests. Chapter 5 describes the data, the recipes, the infrastructure and the evaluation. Chapter 6 gives the results of the first reproduction. Chapter 7 describes the search for the cause of the gap. Chapter 8 uses interpretability methods to find the cause. Chapter 9 tests RoPE in the window branch. Chapter 10 answers the research questions and gives the limits of the study. Chapter 11 summarizes the work and lists the next steps.

CHAPTER 2

Background

This chapter explains the parts that Lizard combines. It uses one notation for all formulas. A sequence has the positions $1, \ldots, L$. At position i, an attention head has the query $\mathbf{q}_i$, and each position t has a key $\mathbf{k}_t$ and a value $\mathbf{v}_t$. The head dimension is d.

2.1 The transformer and causal softmax attention

A decoder-only transformer [27] has a stack of layers. Each layer has an attention block and an MLP block. The attention block projects its input $\mathbf{x}_i$ into queries, keys and values with the matrices W_Q, W_K and W_V. Then each head computes a weighted mean of the earlier values:

$$\mathbf{y}_i = \sum_{t=1}^{i} \frac{\exp\left(\mathbf{q}_i^\top \mathbf{k}_t / \sqrt{d}\right)}{\sum_{j=1}^{i} \exp\left(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}\right)} \mathbf{v}_t. \tag{2.1}$$

The sum stops at i, so a token sees only earlier tokens. This is the *causal* mask. The output projection W_O then mixes the heads. The weights of a row are positive and sum to 1. Thus each output is a weighted mean of values. The weights can be sharp. For example, a head can put almost all weight on the previous token.

The teacher of this thesis is Llama-3.2-1B [7]. It has 16 layers, a hidden size of 2048 and 32 attention heads with a head dimension of 64. The keys and the values use 8 heads, and groups of query heads share them.

2.2 Rotary position embedding

Equation (2.1) has no information about the positions. A transformer must add this information. Llama uses the rotary position embedding (RoPE) [25]. RoPE splits each query and each key into pairs of numbers. It rotates each pair by an angle that grows with the position i. Each pair has its own speed of rotation. Let R_i be the rotation for position i. Then the score becomes

$$\left(R_i \mathbf{q}_i\right)^\top \left(R_t \mathbf{k}_t\right) = \mathbf{q}_i^\top R_{t-i}\, \mathbf{k}_t. \tag{2.2}$$

The rotation matrices are orthogonal, so the score depends only on the distance $t - i$ and not on the absolute positions. Llama-3.2-1B uses the base frequency 500,000. RoPE needs no extra memory. A key gets its rotation one time, at its own position, and the cache keeps the rotated key.

RoPE is important for this thesis. With RoPE, a head can find "the token one

position back" from the positions alone. Without RoPE, a head sees only the content of the tokens.

2.3 The cost of softmax attention

For a sequence of length L, Equation (2.1) computes $L^2/2$ scores in each head. Thus the time grows with L^2. During generation, each new token needs the keys and values of all earlier tokens. Thus the cache grows with L, and the time for each new token also grows with L. Section 1.1 gives the size of the cache for the teacher.

2.4 Linear attention and feature maps

Linear attention [14] replaces $\exp(\mathbf{q}^\top \mathbf{k}/\sqrt{d})$ with a product $\phi(\mathbf{q})^\top \phi(\mathbf{k})$ of a feature map ϕ. The feature map gives positive values. Then the output is

$$\mathbf{y}_i = \frac{\phi(\mathbf{q}_i)^\top \sum_{t=1}^{i} \phi(\mathbf{k}_t)\, \mathbf{v}_t^\top}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^{i} \phi(\mathbf{k}_j)}. \tag{2.3}$$

The two sums do not depend on the query. Thus the model can keep them as a state and add one term for each new token:

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)\, \mathbf{v}_i^\top, \qquad \mathbf{z}_i = \mathbf{z}_{i-1} + \phi(\mathbf{k}_i), \qquad \mathbf{y}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}_i}{\phi(\mathbf{q}_i)^\top \mathbf{z}_i}. \tag{2.4}$$

The state $\mathbf{S}_i$ has the size $f \times d$, with f features. It does not grow with the length. The time for each new token is constant. The price is accuracy: a fixed state cannot store every detail of a long text.

2.5 Hedgehog feature maps

The choice of ϕ decides how well Equation (2.3) can copy softmax attention. Hedgehog [32] learns the feature map. It uses a matrix W and two exponentials:

$$\phi(\mathbf{x}) = [\,\mathrm{softmax}(\mathbf{x}W) \oplus \mathrm{softmax}(-\mathbf{x}W)\,]. \tag{2.5}$$

Here $\oplus$ joins two vectors. With $W \in \mathbb{R}^{d \times f}$, each query and each key gets $2f$ features. All features are positive. The paper of Lizard gives the form with exp in its Section 4, but its Table 13 gives the softmax form. The code of this work uses the softmax form. Hedgehog trains W so that the weights of linear attention look like the weights of softmax attention: positive, monotonic in the score, and sometimes sharp.

2.6 Gated linear attention

Plain linear attention gives the same weight to old tokens and new tokens. Gated linear attention (GLA) [30] adds a gate $\gamma_l \in (0, 1)$ at each position. The gate makes old tokens less important. In the form of Lizard, with one scalar gate for each position, the output is

$$\mathbf{y}_i = \frac{\phi_q(\mathbf{q}_i)^\top \sum_{t=1}^{i} \left(\prod_{l=t+1}^{i} \gamma_l\right) \phi_k(\mathbf{k}_t)\, \mathbf{v}_t^\top}{\phi_q(\mathbf{q}_i)^\top \sum_{j=1}^{i} \left(\prod_{l=j+1}^{i} \gamma_l\right) \phi_k(\mathbf{k}_j)}. \tag{2.6}$$

The recurrent form multiplies the state by the gate before it adds the new token: $\mathbf{S}_i = \gamma_i \mathbf{S}_{i-1} + \phi_k(\mathbf{k}_i)\mathbf{v}_i^\top$. The two forms give the same output (Appendix B). The product of the gates is the weight that a token keeps after some steps. With $\gamma = 0.99$, a token keeps 28% of its weight after 128 steps. With $\gamma = 0.999$, it keeps 88%. The gate is the only source of position information in this branch. It prefers recent tokens, but it cannot select one exact position.

2.7 Sliding window attention and attention sinks

Sliding window attention computes Equation (2.1) over the last w tokens only. Its cost for each token does not grow with the length, and it is exact inside the window. It cannot see tokens before the window.

Softmax models often put a large weight on the first token of a sequence, even when this token has no meaning for the task. Xiao et al. [28] call this token an *attention sink*. A window that removes the first token loses this place for extra weight, and the model gets worse. Lizard adds m learnable sink logits $t_1, \ldots, t_m$ to the denominator of the window:

$$\hat{\mathbf{y}}_i^{\text{win}} = \frac{\sum_{t=i-w+1}^{i} \exp\left(\mathbf{q}_i^\top \mathbf{k}_t / \sqrt{d}\right) \mathbf{v}_t}{\sum_{j=1}^{m} \exp(t_j) + \sum_{t=i-w+1}^{i} \exp\left(\mathbf{q}_i^\top \mathbf{k}_t / \sqrt{d}\right)}. \tag{2.7}$$

The sinks take part of the weight but add no value. Thus the window weights sum to a number $\rho_i \leq 1$. Figure 2.1 shows which keys each type of attention uses.

2.8 Knowledge distillation and attention transfer

In knowledge distillation [12], a student model learns from the outputs of a teacher model. Linearization uses a special form: the student and the teacher have the same weights, except for the attention. Thus each student attention layer can learn directly from the teacher attention layer. The loss compares the two outputs of each layer with the mean squared error (MSE). This is *attention transfer*. Each student layer gets the input of the teacher layer, not the output of the student layer before it. Thus an error in one layer does not change the input of the next layer during this training.

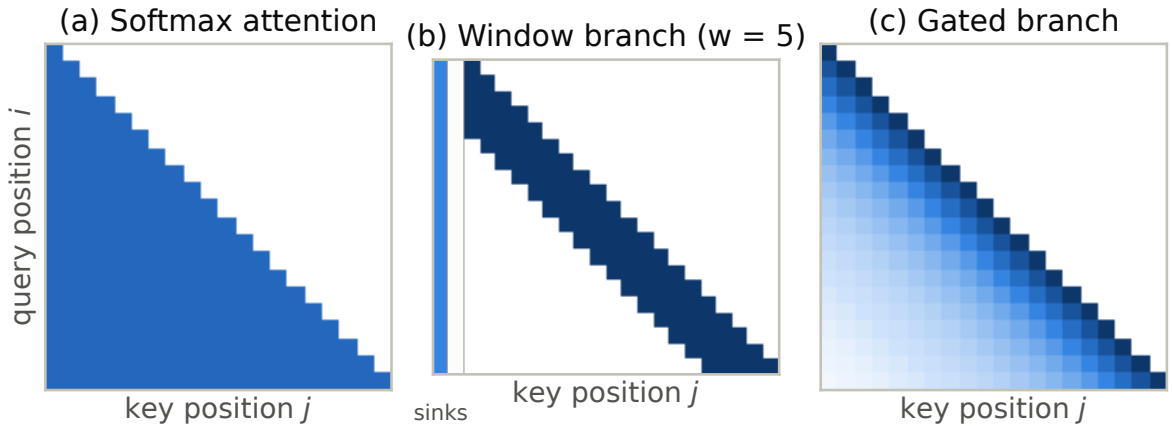


Figure 2.1: Keys that each type of attention uses, in a toy example with 20 positions. Each row is a query and each column is a key. Darker cells have more weight. (a) Softmax attention can use every earlier key. (b) The window branch uses only the last w keys, plus the sink logits on the left. (c) The gated branch uses every earlier key, but the gate makes the weight smaller with the distance. Panel (c) uses a constant gate of 0.82.

2.9 Low-rank adaptation

Attention transfer trains each layer alone. The full model can still lose quality. A second stage trains the whole model with the next-token loss. LoRA [13] makes this stage cheap. It keeps a weight matrix W frozen and adds a product of two small matrices: $W + \frac{\alpha_{\text{LoRA}}}{r} BA$, with the rank r. In this work, $r = 8$ and $\alpha_{\text{LoRA}} = 16$. With LoRA on the four attention projections, stage 2 trains 1,703,936 parameters, which is approximately 0.14% of the model.

2.10 How the harness scores multiple-choice tasks

The evaluation of this thesis does not let the model write an answer. For each question, the harness builds one text for each answer option. It computes the log-likelihood of the option after the question. The option with the highest value counts as the answer of the model. The metric `acc` uses the raw log-likelihood. The metric `acc_norm` divides it by the length of the option text, so that long options are not at a disadvantage.

MMLU uses a different format. The prompt lists four options with the letters A to D and ends with “Answer:”. The harness compares the log-likelihoods of the four tokens “ A” to “ D”. The model must therefore connect its knowledge with the right letter. PIQA and ARC use the full text of each option, so a preference for one letter has no effect there.

2.11 The benchmarks

Table 2.1 lists the benchmarks of this thesis. The paper of Lizard uses the same six benchmarks in its Table 9.

Table 2.1: The benchmarks of this thesis. All tasks are multiple choice. MMLU uses 5 examples before each question (5-shot). The other tasks use no examples (0-shot).

Benchmark	Content	Questions	Options	Metric of the paper
MMLU [11]	Knowledge in 57 subjects, from high school to professional level	14,042	4	`acc`, 5-shot
PIQA [4]	Physical commonsense: a goal and two solutions	1,838	2	`acc`
ARC-Easy [5]	Grade-school science questions, Easy set	2,376	4	`acc`
ARC-Challenge [5]	Grade-school science questions, Challenge set	1,172	4	`acc_norm`
HellaSwag [31]	The most probable end of a short story	10,042	4	`acc_norm`
WinoGrande [22]	Pronoun resolution in a sentence	1,267	2	`acc`

The chance level is 25% for four options and 50% for two options. The teacher is a small model, and it gets only 31.0 on MMLU in the paper. Thus the MMLU scores of all 1B models lie close to chance, and small differences need many questions. Appendix A gives short definitions of further terms, such as packing and padding.

CHAPTER 3

Related Work

3.1 Linear attention and recurrent models

Katharopoulos et al. [14] showed that linear attention is a recurrent network with a matrix state. Later work added gates and better feature maps. GLA [30] uses a gate for each key dimension and trains models from the start. Mamba-2 [6] connects state space models with linear attention. Based [1] joins linear attention with a small sliding window. The window gives exact local attention, and the linear part gives the long range. Lizard uses the same idea. These models need a full pretraining run. Linearization avoids this cost.

3.2 Linearization methods: LoLCATs, Liger and Lizard

LoLCATs [33] trains Hedgehog feature maps to copy the attention of a teacher (attention transfer), then trains LoRA weights. Its window has RoPE, and its linear part covers only the tokens before the window. The window and the linear part share one denominator. On Llama-3-8B, LoLCATs with 50% softmax layers gets 65.8 on MMLU, against 52.8 for the fully linear model and 66.6 for the teacher.

Liger [18] has no attention transfer stage and adds no new parameters. It uses the teacher projections in a GLA form with a fixed feature map. It mixes a gated branch and a window with the factor 0.5 each. The paper writes the window without RoPE, but the code of Liger applies RoPE. Liger trains only LoRA weights with the next-token loss.

Lizard [20] keeps the attention transfer stage of LoLCATs. It adds a learned scalar gate, four sink logits and the weight α. Its window has no RoPE by design. The paper wants the gate to learn the relative position, so that the model can work on inputs longer than in training. Table 3.1 compares the three methods with GLA. Section 4.10 gives the formulas.

3.3 Linearized models at the 1B scale

Other papers also report weak MMLU scores for linearized 1B models. Table 3.2 collects the numbers that the project notes found. The Liger paper reports 23.1 for LoLCATs and 22.4 for Liger on Llama-3.2-1B. The Lizard paper reports 27.3 for LoLCATs on

Table 3.1: Four attention methods side by side. The values come from the papers and the code of each method (project notes, document 15).

Part	Lizard	LoLCATs	Liger	GLA
Feature map	Hedgehog, learned, shared by the heads	Hedgehog, learned, one for each head	Softmax over the head dimension, no weights	None
Gate	One scalar for each token	None	From the key, power 1/16 (code)	One value for each key dimension
Window	128 keys, 4 sinks, no RoPE	64 keys, RoPE	64 keys, RoPE in the code	None
Tokens in the linear part	All earlier tokens	Only tokens before the window	All earlier tokens	All earlier tokens
Denominator	Own row sum for each branch	Shared by both parts	None in the linear part	None
Total weight of a row	$1 + \alpha\rho_i$	1	$0.5 + 0.5\mu_i$	–
Training	Attention transfer, then LoRA	Attention transfer, then LoRA	LoRA only	Pretraining

the same model. Thus two papers give different values for the same method, and the evaluation details matter. STILL [23] keeps exact softmax attention over selected tokens and reports 29.8.

Table 3.2: Reported results of linearized Llama-3.2-1B models. Each paper uses its own harness. The values are accuracies in %.

Source	Model	MMLU	PIQA	ARC-Easy
Lizard paper, Table 9	Teacher	31.0	74.1	65.4
	LoLCATs	27.3	74.6	63.0
	Lizard	29.8	74.8	65.6
Liger paper, Table 4	Teacher	31.0	–	–
	LoLCATs	23.1	74.1	63.7
	Liger	22.4	75.0	65.4
STILL	STILL	29.8	–	–

Two more results connect to this thesis. RADLADS [9] uses a second stage with the Kullback–Leibler (KL) divergence to the teacher logits, not the next-token loss on new data. A report with the title “Stuck on A” [24] describes a linearized 0.6B model with a perplexity near the teacher. But the model has chance accuracy on multiple-choice tasks, and it gives the answer “A” for 81% of the questions. Chapter 6 describes the

same symptom.

3.4 Hybrid models and retrieval heads

Several methods keep a small part of the softmax attention. Bick et al. [3] study *gather-and-aggregate* (G&A) heads. A few such heads do the letter step of MMLU. In a pruned Llama-3.1-8B, one disabled G&A head drops MMLU from 66% to 25%. Recurrent models give smoother attention patterns, without the sharp attention that these heads need. DuoAttention [29] keeps full attention for a few retrieval heads and uses a window with sinks for the other heads. A KL-guided method [17] selects which layers stay softmax. LoLA [19] adds a small cache of tokens that linear attention cannot recall. These ideas are options for the MMLU gap (Chapter 11).

3.5 Interpretability of distilled attention

The LoLCATs paper plots the error of each layer after attention transfer (its Figure 14). It also compares the attention weights of the teacher and the student (its Figures 18–21). Chapter 8 uses both methods. Further methods exist, for example activation patching [10], induction heads [21] and the tuned lens [2]. This thesis does not use them, but Chapter 11 lists them as next steps.

CHAPTER 4

Lizard and Its Implementation

4.1 Lizard attention

Lizard [20] replaces every softmax attention layer of the teacher with two branches (Figure 4.1). The teacher projections W_Q, W_K, W_V and W_O stay as they are. Lizard adds five small parameters to each layer.

Gated branch

Gated linear attention (Equation (2.6)) with Hedgehog feature maps ϕ_q and ϕ_k (Equation (2.5)). The gate is one scalar for each token, $\gamma_i = \sigma(W_\gamma \mathbf{x}_i)$. All 32 heads of a layer share the feature maps and the gate. The branch divides by its own row sum, so its weights sum to 1.

Window branch

Softmax attention over the last $w = 128$ keys, with $m = 4$ sink logits in the denominator (Equation (2.7)). The paper calls it *anchor window attention*. It has no RoPE.

Output

The two branches add up: $\hat{\mathbf{y}}_i = \hat{\mathbf{y}}_i^{\text{gate}} + \alpha\, \hat{\mathbf{y}}_i^{\text{win}}$, with one learnable α for each layer.

Table 4.1 lists the parameters of one layer. Lizard adds 18,437 parameters to each layer and 294,992 to the 16 layers. This is 0.024% of the model.

Table 4.1: The Lizard parameters of one layer, with feature dimension 128 (project notes, document 1).

Parameter	Shape	Count	Function
`phi_q`, `phi_k`	64×128 each	16,384	Feature maps with 256 features. All heads share them.
`W_gamma`	2048×1	2,048	Gate. Start value 0, so $\gamma = 0.5$ at the start.
`meta_tokens`	4	4	Sink logits in the window denominator
`alpha_blend`	1	1	Weight α of the window branch. Start value 1.
Total		18,437	

4.2 The two training stages

Figure 4.2 shows the two stages as the paper describes them.

Stage 1: attention transfer. Only the Lizard parameters train. Each Lizard layer gets the input of the teacher layer. The teacher computes its own output with

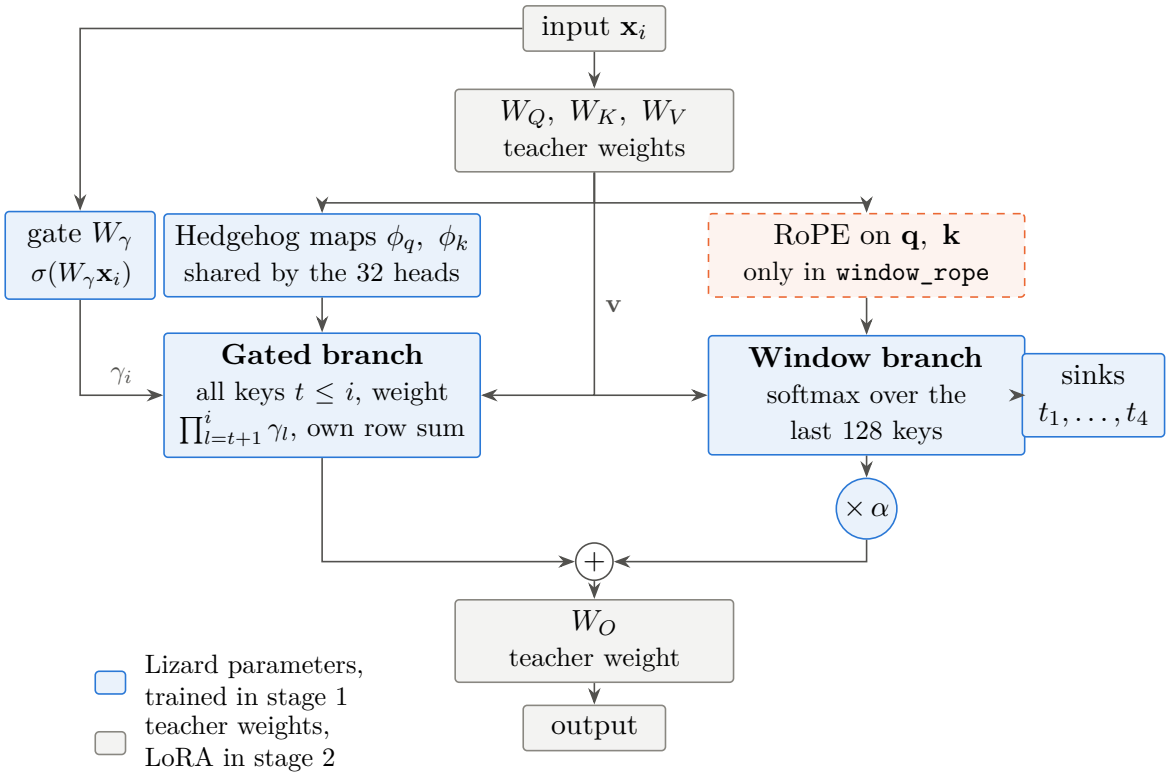


Figure 4.1: One Lizard attention layer. The teacher projections give the queries $\mathbf{q}$, the keys $\mathbf{k}$ and the values $\mathbf{v}$. Blue parts are the Lizard parameters. Gray parts are the weights of the teacher. The orange RoPE box is the change of Chapter 9. The original Lizard has no RoPE in either branch.

RoPE. The loss compares the two outputs of each layer:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{l=1}^{N} \left\| \mathbf{Y}^{l}_{\text{softmax}} - \hat{\mathbf{Y}}^{l}_{\text{Lizard}} \right\|_F^2 . \tag{4.1}$$

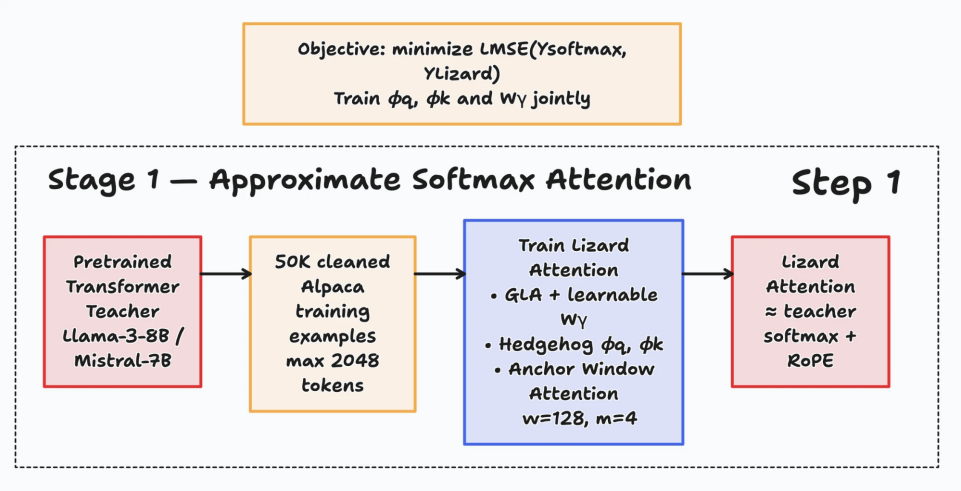
The code of this work uses 1000 times the mean squared error of each layer. This form differs from Equation (4.1) only by a constant factor, and the Adam optimizer does not change with such a factor.

Stage 2: fine-tuning. LoRA weights on the attention projections train with the next-token loss. In the code of this work, the Lizard parameters stay frozen in stage 2. The paper does not say if they train.

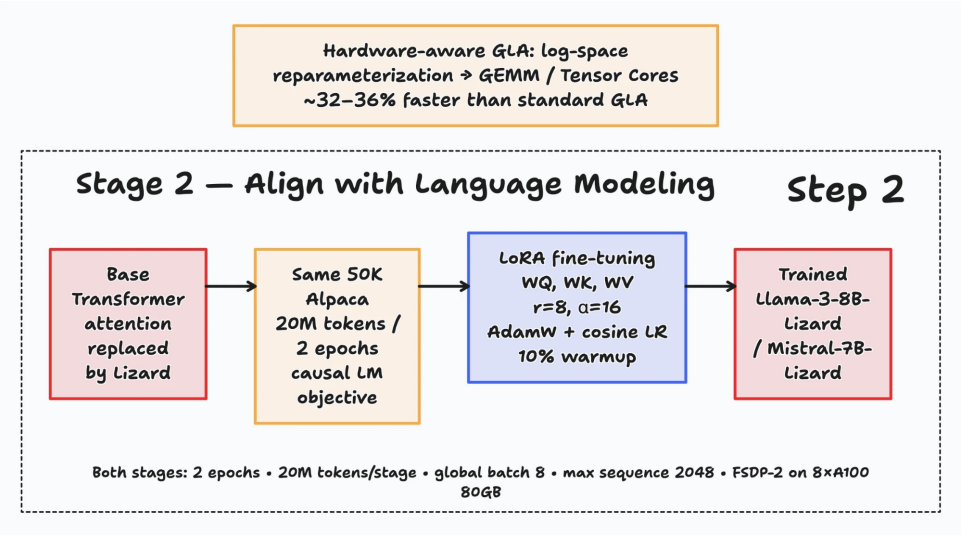
4.3 The training algorithm in log space

Section 4 of the paper gives a fast algorithm for the gated branch. In matrix form, the branch needs the cumulative gate products $c_t = \prod_{j \le t} \gamma_j$ and the quotients c_i / c_t. The products get very small, and bf16 cannot store them. The paper moves $\log c_t$ into the exponent of the feature maps, so that the branch becomes one matrix product.

A proof audit of this work found a limit (Table 4.5, P3). The step holds for the



(a) Stage 1: attention transfer with the MSE loss.



(b) Stage 2: LoRA with the next-token loss.

Figure 4.2: The two training stages of Lizard, as the paper describes them. The paper uses 8B teachers in this description. This thesis uses Llama-3.2-1B. Source: the figures of the project notes, drawn by the author.

exponential feature map. With the softmax feature map of Table 13, the term $\log c_t$ cancels in the softmax, and the gate disappears. Also, $1/c_t$ overflows after approximately 126 tokens at $\gamma = 0.5$, so only a form that works in chunks can use it. This work does not use the fast algorithm. It computes the parallel form of Equation (2.6) in float32 and materializes the full $L \times L$ matrix.

4.4 The reference implementation

The work started with a reference implementation in the repository `jku-thesis` [15]. It is pure PyTorch. The functions `hedgehog`, `gla` and `awa` implement the formulas directly, and a class `LizardAttention` joins them. A second file contains slow loop and

recurrent versions of every formula. These versions are the ground truth. The tests compare the two versions in float64 and require a relative error below 10^{-10}. Further adapters compare the branches with external implementations, for example the GLA kernels of the `flash-linear-attention` library and PyTorch attention with a sink column.

4.5 The port to LoLCATs

The training of this work uses the LoLCATs code base [16, 33]. It already has the two stages, the data pipeline and the evaluation. The functions of the reference code went into LoLCATs without changes. A wrapper class connects them with the LoLCATs model:

- It takes the teacher attention module and uses its projections again. Thus only the five Lizard parameters are new.
- In stage 1, it computes the teacher output with RoPE as the target. It returns the student output and the target to the trainer. The next layer gets the teacher output.
- In evaluation and generation, it uses a cache. The cache keeps the state of the gated branch and the keys and values of the last 128 tokens. A prompt runs in one dense pass. Each new token then uses the recurrent form.

Table 4.2 shows the differences between the two code bases. The LoLCATs recipe became the first recipe of this work, because the task was to keep the existing configs.

Table 4.2: The reference pipeline (`jku-thesis`) against the LoLCATs pipeline of the first runs (project notes, document 1).

Aspect	Reference pipeline	LoLCATs pipeline
Stage 1 loss	Summed squared error for each layer	1000 × mean squared error for each layer
Stage 2 parameters	LoRA on q, k, v and the Lizard parameters	LoRA on q, k, v, o. Lizard parameters frozen.
Precision	float32	bf16 model, Lizard computations in float32
Recipe	The recipe of the paper	The LoLCATs configs: learning rate 10^{-2} in stage 1 and 10^{-4} in stage 2

4.6 Tests of the attention layer

The poor first results (Chapter 6) raised the question of a bug. Two levels of tests ran on CPU.

Level 1: the formulas. A small module made the 28 tests of the reference code run against the LoLCATs functions. 26 tests passed. The two other tests need a newer version of the `transformers` library and fail for the same reason on the reference code. The tests show these properties: the feature maps are positive, and each half sums to 1. The gated branch matches the loop form and the recurrent form. The window with sinks matches its loop form. Future tokens give no gradient. The gradients match finite differences.

Level 2: the model. A tiny Llama with random weights in float64 tested the wrapper, the conversion of the model and the cached paths. The output with the cache equals the dense output up to float64 rounding.

> The tests show that the code computes the equations of the paper, and that the evaluation path computes the same function as the training path. They do not test the quality of the trained weights.

4.7 The paper against the code

A line-by-line comparison of the formulas of the paper with the code found 17 differences. Most of them have no effect. Table 4.3 lists the points where the paper allows more than one reading.

Table 4.3: Open points of the paper and the choice of the code (project notes, documents 9, 13 and "math against code").

Point	What the paper says	Choice of the code
D1: normalization of the gated branch	The parallel form has a denominator. The recurrent form and the matrix form have none.	Denominator (row sum 1)
D2: feature maps for each head	Not given. The LoLCATs default has one map for each head.	One map for all heads of a layer
D3: activation	Section 4 gives exp. Table 13 gives softmax.	Softmax
D5: sink term	The equation has $\sum_j t_j$, but the text calls t_j a logit.	$\sum_j \exp(t_j)$
Gate start value	Not given	$W_\gamma = 0$, so $\gamma = 0.5$
α	Not listed as a learnable parameter in the loss	Learnable, start value 1

The written sink term $\sum_j t_j$ can make the denominator zero or negative (Table 4.5, P4). Thus the code follows the word "logit".

4.8 Lizard attention v2: one option for each reading

A second attention file (v2) makes each open point an option. With the default options, v2 gives exactly the outputs of v1. Table 4.4 lists the options.

Table 4.4: The options of Lizard attention v2. The first value is the default (project notes, document 13).

Option	Values	Meaning
`gla_norm`	`row`, `none`, `joint`, `hybrid`	Row sum of the gated branch (D1). `joint` and `hybrid` use one denominator for both branches and the sinks. `hybrid` is the LoLCATs form.
`alpha_per_head`	false, true	One α for each head
`alpha_init`	1.0, any number	Start value of α
`feature_map_per_head`	false, true	One feature map for each head (D2)
`feature_activation`	softmax, exp	D3
`gate_per_head`	false, true	One gate for each head
`gate_bias_init`	none, a number	A bias in the gate with a start value
`window_rope`	false, true	RoPE in the window branch only

4.9 Proofs and float64 tests

The options depend on claims about the formulas. A proof audit checked each claim by hand. A float64 script then computed the smallest cases. Table 4.5 lists the main claims.

Three claims matter most for the later chapters. P8 shows that a Lizard row is not a weighted mean of values: its weights sum to more than 1. P2 shows that α limits how much a head can take from the exact window. P10 shows that the window without RoPE cannot find the previous token from the positions.

4.10 Lizard, LoLCATs, GLA and Liger in formulas

With the notation $\kappa_{it} = \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_t)$, the gate products c_{it} and the window scores s_{it}, the outputs of Lizard and LoLCATs are

$$\text{Lizard:} \quad \mathbf{y}_i = \frac{\sum_{t \le i} c_{it}\, \kappa_{it}\, \mathbf{v}_t}{\sum_{t \le i} c_{it}\, \kappa_{it}} + \alpha\, \frac{\sum_{t \in W_i} e^{s_{it}} \mathbf{v}_t}{\sum_j e^{t_j} + \sum_{t \in W_i} e^{s_{it}}}, \tag{4.2}$$

$$\text{LoLCATs:} \quad \mathbf{y}_i = \frac{\sum_{t \in W_i} a\, e^{s_{it}} \mathbf{v}_t + \sum_{t \le i-w} \kappa_{it}\, \mathbf{v}_t}{\sum_{t \in W_i} a\, e^{s_{it}} + \sum_{t \le i-w} \kappa_{it}}. \tag{4.3}$$

LoLCATs gives an exact weighted mean of the values. Its linear part excludes the window tokens. Lizard counts the window tokens twice, one time in each branch (claim

Table 4.5: Main claims of the proof audits (project notes, documents 13, 14 and 15). "Restricted" means: true only under the stated condition.

Claim	Statement	Verdict
P1	The recurrent form equals the parallel form, without and with the denominator	Proved
P2	The window share of a head is at most $\alpha/(1+\alpha)$	Restricted: $\alpha \geq 0$
P3	The log-space form equals the gated matrix form	Only for the exp map
P4	The written sink term $\sum_j t_j$ can make the denominator 0 or negative	Proved
P5	The decode forms of `joint` and `window_rope` equal their parallel forms	Proved
P6	`joint` keeps a usable share for the gated branch at the start	Refuted
P7 (H)	Four sink logits with equal start values stay equal, so they act as one sink	Proved
P8 (C)	The total weight of a Lizard row is $1 + \alpha\rho_i$, more than 1	Proved
P9	With `hybrid` and $\gamma = 1$, the window share falls as $1/i$	Restricted
P10 (F)	A window without RoPE cannot give exact previous-token attention in layer 0	Restricted: no position input

B of document 15). The `hybrid` option of v2 equals LoLCATs under three conditions (claim A). All gates are 1, there are no sinks, and the linear sums cover only $t \leq i - w$.

4.11 Numerical stability: the NaN crash

The first full run stopped in stage 2 at gradient step 386. The forward pass gave NaN, and a debugger breakpoint in the trainer then ended the job. A review found two operations without protection:

- The window branch computed $\exp(\mathbf{q}^\top \mathbf{k}/\sqrt{d})$ directly. In float32, exp overflows above approximately 88.7, and ∞/∞ gives NaN. LoRA trains the query and key projections in stage 2, so the scores can grow.
- The gated branch divides by the row sum. If all weights of a row underflow to 0, the result is 0/0.

The fix subtracts the largest score and the largest sink logit before exp, and it gives the row sum a small lower limit. Both changes keep the formulas the same. The overflow is the most probable cause, but no test reproduced it on the real checkpoint.

CHAPTER 5

Experimental Setup

5.1 The teacher and the data

The teacher is `meta-llama/Llama-3.2-1B` in bf16, with the model sizes of Section 2.1. Both stages use the cleaned Alpaca data set [26] (`yahma/alpaca-cleaned`), as the paper does. The pipeline joins the examples and cuts the text into sequences of 2048 tokens. This is *packing* (Appendix A). Table 5.1 gives the sizes.

Table 5.1: Data of both training stages (measured in the first run, project notes, document 2).

Quantity	Value
Training examples	51,560
Validation examples	200
Sequences of 2048 tokens in each epoch	4,714 (approximately 9.7 million tokens)
Epochs in each stage	2
Sequences in each gradient step	8 (batch 1 × 8 accumulation steps)
Gradient steps in each stage	1,178
Validation	every 100 steps

5.2 The training recipes

The first runs used the existing LoLCATs configs. Their recipe differs from the recipe of the paper in several points (Table 5.2). The most important differences are the learning rate of stage 1, the schedule and the gradient clipping. Chapter 7 tests these differences one at a time.

5.3 Infrastructure

Figure 5.1 shows the path from a code change to a result. A GitHub workflow builds a Docker image with the training code. Hugging Face Jobs runs the image on one rented GPU. The job pushes the checkpoints to a private model repository on the Hugging Face Hub. A university machine with one NVIDIA A10 (24 GB) then downloads the checkpoints and runs the evaluation. W&B logs the training curves.

Some problems occurred on this path. The first evaluation job failed because the Python import path did not contain the repository root. Several scripts used fixed paths under `/workspace`. The W&B logger ignored all stage 2 metrics, because the

Table 5.2: The LoLCATs recipe of the first runs against the recipe of the paper. Sources: Section 5 and Table 13 of the paper, project notes, documents 4 and 9. Bold rows are the largest differences.

Setting	Paper	First runs (LoLCATs configs)
Data	Alpaca-cleaned, 2 epochs, 2048 tokens	The same
Global batch	8 (micro batch 1)	The same
Precision	bf16	bf16 model, Lizard computations in float32
Learning rate, stage 1	**10^{-3}**	**10^{-2} (10× higher)**
Learning rate, stage 2	5×10^{-4}	10^{-4} (5× lower)
Schedule	**Cosine, 10% warmup, minimum 0.1× peak**	**Almost constant (reduce on plateau), no warmup**
Gradient clipping	**1.0**	**None**
AdamW β	(0.9, 0.99)	(0.9, 0.999)
LoRA	$r = 8$, $\alpha = 16$, on W_Q, W_K, W_V	$r = 8$, $\alpha = 16$, on q, k, v and o
Lizard parameters in stage 2	Not given	Frozen
Checkpoint	Not given	Best validation loss

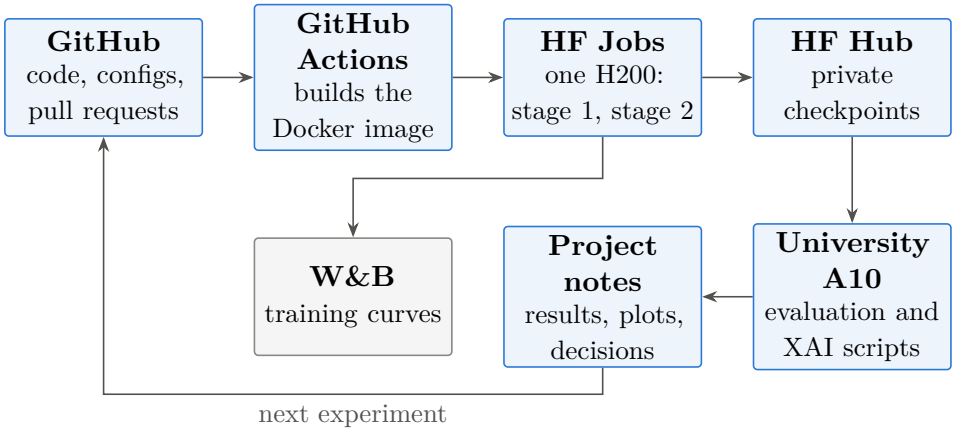


Figure 5.1: The path from a code change to a result. HF is short for Hugging Face. Each experiment is a pull request with a config. A new Docker image is necessary when the code or the config changes.

step counter started again at 0. A script `eval.sh` fixed the evaluation for any machine with an NVIDIA GPU and conda. A second script, `compare_stages.sh`, evaluates the teacher, the stage 1 model and the stage 2 model with the same tasks. It also checks that every trainable parameter loads from its checkpoint.

5.4 Compute and cost

The Lizard code of this work is a dense reference. It materializes full $L \times L$ matrices in float32 for both branches. Thus a 1B model needs much memory. A measurement

on CPU gave approximately 22–25 GB for stage 1 and 41–46 GB for stage 2, with sequences of 2048 tokens. Stage 2 sets the GPU size, because the gradients go back through all layers. Table 5.3 gives the measured times and costs.

Table 5.3: Measured time and cost (project notes, documents 2, 13 and the MMLU summary). The H200 cost $5.00 for each hour.

Task	Time
Stage 1 on the H200, bf16 (approximately 3.2 sequences each second)	approximately 50 minutes
Stage 2 on the H200, bf16 (approximately 1.07 seconds for each sequence)	approximately 2 hours 50 minutes
Full run of both stages, with set-up and uploads	approximately 3 hours 50 minutes, $19
Stage 1 on the H200 in float32	approximately 2.5 hours
PIQA, full Lizard model, on the A10 (3,676 requests)	3 minutes 11 seconds
MMLU, all questions, teacher, on the A10	2 hours 30 minutes
MMLU, all questions, `window_rope`, on the A10	4 hours 31 minutes

The project selected the H200 over the A100. The H200 was approximately 2.7 times faster in the estimate and 20–25% cheaper for a full run, although its price for each hour is two times higher. One reason is that the float32 matrix products of the dense code run at the plain FP32 speed of the A100. Stage 2 took approximately two times the estimated time. A suspected cause is a copy of the full model output from the GPU to the CPU at every step in the LoLCATs trainer.

5.5 The evaluation

All evaluations use the LM Evaluation Harness [8] at commit `b281b09`, as the LoLCATs repository specifies. Table 5.4 lists the facts about this version that affect the results.

Table 5.4: Facts about the harness that affect the results (project notes, document 6).

Fact	Effect
The MMLU score is the unweighted mean over the 57 subjects.	It differs a little from the mean over all questions.
Each MMLU question gives 4 log-likelihood requests, one for each letter.	14,042 questions give approximately 56,000 requests.
`-limit N` takes the first N questions of each subject after a shuffle with seed 42.	All models get the same questions. A paired comparison is possible.
The batch size is always 1.	The evaluation is slow, but no memory errors occur.
The harness adds no beginning-of-text token for Llama.	The teacher and the Lizard models get the same input. The absolute scores can differ from other harness versions.

The *MMLU subset* has 5 questions from each subject, thus 285 questions. Its standard error is approximately 2.6 points. It is useful for fast checks, but not for small differences. The final MMLU values of this thesis use all 14,042 questions.

5.6 Statistics

The harness reports the binomial standard error of an accuracy p on n questions:

$$\mathrm{SE} = \sqrt{p\,(1-p)/n}. \tag{5.1}$$

For PIQA ($n = 1{,}838$), the SE is approximately 1.1 points. For ARC-Easy ($n = 2{,}376$), it is approximately 1.0 point. Most comparisons in this thesis are unpaired. They use $z = (p_1 - p_2)/\sqrt{\mathrm{SE}_1^2 + \mathrm{SE}_2^2}$. A difference counts as clear if $|z| \geq 2$. For all MMLU questions, the answers of each question are available for two models. Then a paired SE and McNemar's test are possible (Section 9.7). Each training run has one random seed. Thus the variance from run to run is not known (Section 10.4).

5.7 The teacher in this harness

The paper used a newer version of the harness. Thus the real target is the teacher in the harness of this work. Table 5.5 shows that the two harness versions give almost the same teacher scores on PIQA and ARC-Easy.

Table 5.5: The teacher in this harness and in the paper. Sources: project notes, document 7 and the teacher summary. Accuracy in %.

Task	This harness	Paper (Table 9)
PIQA	74.4 ± 1.0	74.1
ARC-Easy	65.3 ± 1.0	65.4
MMLU, all questions	31.7 ± 0.4	31.0
MMLU subset (285 questions)	33.7	–

5.8 The models of this thesis

Table 5.6 lists the main models. Appendix D lists every run with its loss and its scores.

Table 5.6: The main models of this thesis.

Name	Description
Teacher	`meta-llama/Llama-3.2-1B`, unchanged
First Lizard model	Lizard v1, feature dimension 128, LoLCATs recipe. Stage 1 of Run 1 and stage 2 of Run 2.
Config 1, config 2	Stage 1 with the recipe of the paper in float32, feature dimension 32. Config 2 adds gradient clipping.
C1, R1, R1b	Stage 1 with options of v2: one α for each head (C1), and per-head maps with a shared denominator (R1, R1b)
`window_rope`	Lizard v2 with RoPE in the window branch. Otherwise the setup of Run 1.
`window_rope` + `hybrid`	`window_rope` with the shared denominator of LoLCATs
LoLCATs control	The LoLCATs attention in the same pipeline, stage 1 only

CHAPTER 6

Results of the Reproduction

This chapter answers RQ1 for the first Lizard model. It uses the LoLCATs recipe of Table 5.2 and the original Lizard attention.

6.1 Runs 0, 1 and 2

Run 0 was a smoke test with 10 steps in each stage. It passed from the start to the end. The stage 1 validation loss fell from 76.9 to 17.1 in 10 steps, and the job pushed the checkpoints to the Hub.

Run 1 was the first full run. Stage 1 finished all 1,178 steps with a best validation loss of 3.2549. Stage 2 then started from a much better model than in the smoke test. Its first validation loss was 5.42 (perplexity 233). After only 10 stage 1 steps, it was 8.85 (perplexity 7,207). Thus stage 1 had a clear effect. At gradient step 386 of stage 2, the forward pass gave NaN, and the job stopped (Section 4.11).

Run 2 repeated stage 2 from the stage 1 checkpoint of Run 1, with the NaN fix. It finished with a validation loss of 2.252 (perplexity approximately 9.5). The *first Lizard model* of this thesis is the stage 1 checkpoint of Run 1 with the stage 2 checkpoint of Run 2.

6.2 The scores against the paper and the teacher

Table 6.1 and Figure 6.1 compare the first Lizard model with the teacher and with the paper. On all three tasks, the model is clearly below the paper. On PIQA, it is approximately 6.3 SE below the paper, and on ARC-Easy, 10.6 SE. On MMLU, it is 4.8 SE *below* chance.

Table 6.1: Scores of the first Lizard model (project notes, document 7). Accuracy in %, with the SE where the harness reports it.

Task	First Lizard model	Teacher, this harness	Lizard, paper	LoLCATs, paper	Teacher, paper
MMLU, all questions	**23.3**	31.7	29.8	27.3	31.0
MMLU subset	24.9	33.7	–	–	–
PIQA	**68.0** ± 1.1	74.4 ± 1.0	74.8	74.6	74.1
ARC-Easy	**54.8** ± 1.0	65.3 ± 1.0	65.6	63.0	65.4

PIQA and ARC-Easy are far above chance (50% and 25%). Thus the model still works as a language model. But it is 6.5 points below the teacher on PIQA and 10.5

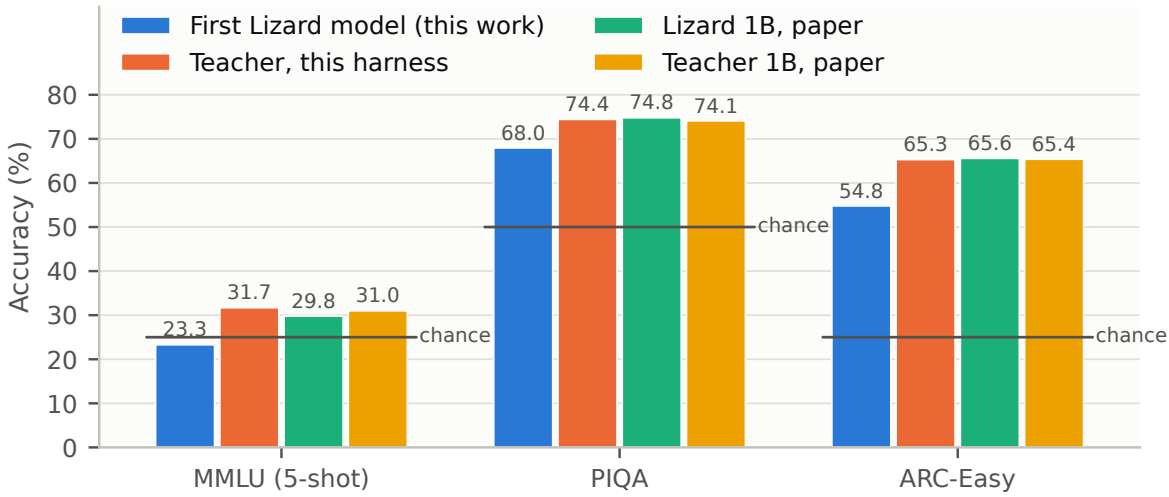


Figure 6.1: The first Lizard model against the teacher and the paper. The dark lines show the chance level of each task. Values from Table 6.1.

points on ARC-Easy. The damage is wide. It is not limited to MMLU or to long texts: most PIQA and ARC-Easy prompts are shorter than the 128-token window.

6.3 The model selects "A" on MMLU

A score below chance on questions with four options suggests a simple behavior: the model ignores the question and prefers one letter. Then its accuracy in each subject equals the share of that letter among the right answers. Table 6.2 tests this idea for each letter.

Table 6.2: Accuracy of the first Lizard model against fixed answers on all MMLU questions (`letters.py`, project notes, document 7). The real macro mean is 0.233.

If the model always gave	Macro mean	Mean |acc − share| over subjects	Correlation with the accuracy
"A"	**0.231**	**0.004**	**+0.98**
"B"	0.245	0.039	+0.29
"C"	0.254	0.051	−0.05
"D"	0.270	0.087	−0.65

> On MMLU, the first Lizard model selects "A" for almost every question. On the MMLU subset, it gives "A" for 98.6% of the questions. The accuracy of each subject has a correlation of +0.98 with the share of "A" as the right answer.

The question and the four options are usually in the last approximately 100 tokens, inside the window. Thus the collapse is not only a failure of long contexts. The teacher itself is only 6–9 points above chance on MMLU. Thus a model with moderate damage can already fall back to one letter.

6.4 The model uses both branches

An earlier idea was that the gated branch did not learn and that the model used only the window. A test removed one branch at inference time and multiplied the other branch by $(1 + \alpha)$. Table 6.3 gives the result on PIQA.

Table 6.3: PIQA with one branch removed at inference (`ablate.py`, project notes, document 7).

Model	Accuracy (%)	Difference
Full first Lizard model	67.95 ± 1.09	–
Window branch removed: $(1 + \alpha)\cdot$ gated branch	57.18 ± 1.15	−10.8 ($z \approx 6.8$)
Gated branch removed: $(1 + \alpha)\cdot$ window branch	52.34 ± 1.17	−15.6 ($z \approx 9.8$)
Chance	50.0	

Both branches matter. Without the gated branch, the model is almost at chance. This agrees with the design of the paper. Without RoPE, the window cannot find the token order. The gate is the only source of position information. The test measures how much the trained model depends on each branch. It does not show if the architecture needs each branch. The paper tests that question with new training runs (its Table 6).

6.5 The gap starts in stage 1

The script `compare_stages.sh` evaluated the model after stage 1, without LoRA, and after stage 2. Table 6.4 shows the result. The drop is already present after stage 1. Stage 2 recovers approximately 10 points on PIQA, but MMLU stays near chance. The collapse to "A" is also present after stage 1. All checkpoints loaded completely: 80 Lizard tensors and 128 LoRA tensors, with 0 missing keys. Thus a loading error does not cause the results.

Table 6.4: The first Lizard model after each stage (project notes, "stage difference").

Task	After stage 1	After stage 2	Teacher
PIQA	57.6 ± 1.2	68.0 ± 1.1	74.4
ARC-Easy	39.0 ± 1.0	54.8 ± 1.0	65.3
MMLU subset	22.5 ("A" for 95.4%)	24.9 ("A" for 98.6%)	33.7

The gate values give a first explanation. The script measured the gates on one MMLU prompt of 2048 tokens. In layers 1–15, the gate is above 0.999 for all tokens. After 512 tokens, these layers keep at least 95% of the weight of a token. Thus the gated branch adds up almost all earlier tokens without decay. Only layer 0 has a decay: it keeps 35% of the weight after 128 tokens. In layer 0, α fell from 1 to 0.042. Thus the window branch of layer 0 has almost no weight. The other layers have α between

0.16 and 0.65. Most sink logits stopped at exact powers of two, such as 0.5, 1.0 and 2.0. This is a sign of bf16 rounding.

6.6 The temperature does not change the scores

The harness compares log-likelihoods. A temperature T divides all logits by the same number. This does not change the order of the four letters, so the accuracy must stay the same. Table 6.5 confirms this prediction on the stage 1 model.

Table 6.5: The MMLU subset at four temperatures, stage 1 model (project notes, "temperature"). "Mass" is the probability of the four letters together. "Changed" counts the answers that differ from $T = 1$.

T	Accuracy	"A"	Changed	Mass	Entropy (bits)
0.1	22.5	95.4%	0 of 285	0.006	0.070
0.5	22.5	95.4%	0 of 285	0.025	0.855
1	22.5	95.4%	0 of 285	0.018	1.550
2	22.5	95.4%	0 of 285	0.002	1.879

The test also shows a second problem. The four letters together get only 1.8% of the probability of the next token at $T = 1$. Thus, for almost all questions, the most probable next token after "Answer:" is not a letter. The stage 1 model does not follow the answer format of the 5-shot prompt.

6.7 Summary of the reproduction

RQ1, first answer. The first reproduction did not reach the paper. MMLU is 23.3 against 29.8, PIQA 68.0 against 74.8 and ARC-Easy 54.8 against 65.6. The teacher in the same harness gets the expected scores, and the tests of Section 4.6 found no bug. The drop starts in stage 1. On MMLU, the model gives "A" for almost every question.

CHAPTER 7

The Search for the Cause of the Gap

This chapter addresses RQ2. It tests the possible causes of the gap one at a time. Figure 7.1 shows the order of the work.

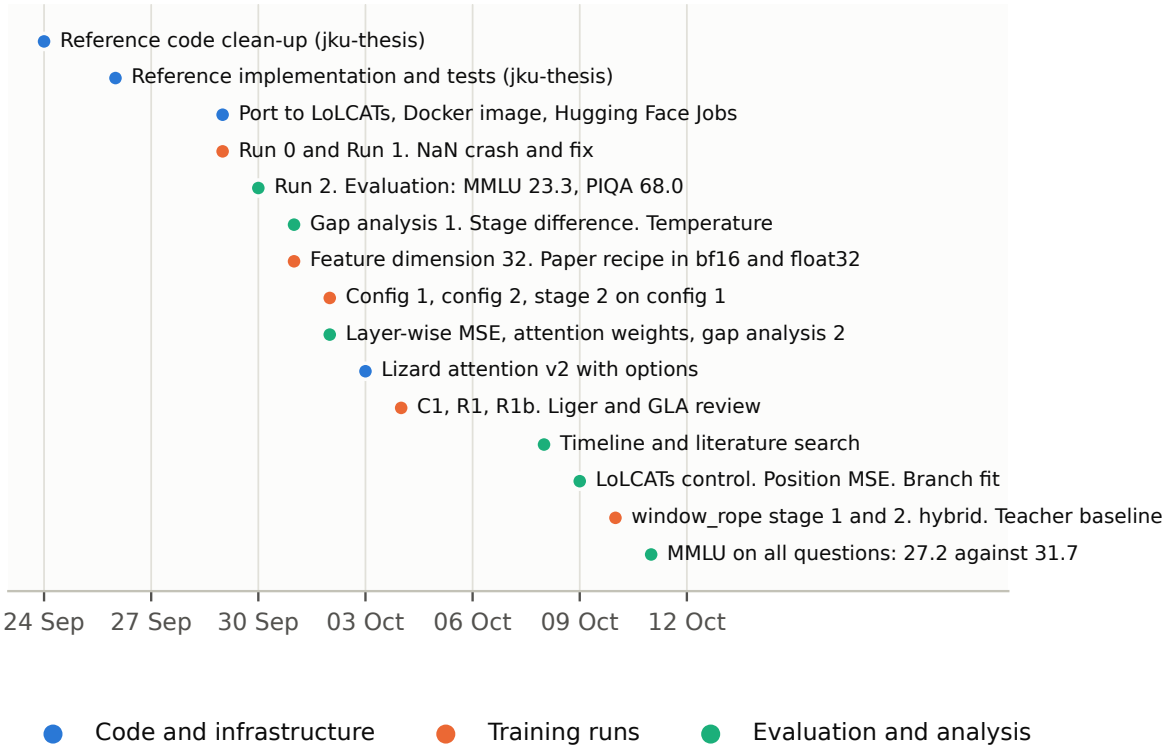


Figure 7.1: Timeline of the project, from the pull requests of the two repositories and the project notes (document 18).

7.1 The possible causes

The tests of Section 4.6 excluded a bug in the formulas. The first gap analysis ranked the remaining causes (Table 7.1). The recipe had the largest difference from the paper, so it came first. A missing checkpoint key was cheap to check, so it came next.

7.2 Precision: bf16 can discard small updates

The first runs store the model and the trainable weights in bf16. This format keeps only approximately 3 significant digits. An AdamW step changes a weight by approximately the learning rate. If the step is smaller than half the distance between two neighboring bf16 numbers, rounding discards it. At the stage 1 learning rate of LoLCATs (10^{-2}), rounding discards no updates. At the stage 2 learning rate (10^{-4}), it discards 43–65%

Table 7.1: Ranking of the possible causes after the first reproduction (project notes, document 11), and the later result of each test.

#	Possible cause	Result of the test
1	The stage 1 recipe differs from the paper	Not the main cause (Section 7.3)
2	The stage 2 recipe and the frozen Lizard parameters	No change in accuracy (Section 7.4)
3	Missing keys when the checkpoint loads	Excluded: 0 missing keys in every evaluation
4	The data, the packing or the prompts	Not tested. The data and the packing follow the paper.
5	Numerical effects of bf16	Real for the recipe of the paper (Section 7.2)
6	The harness version	Small: the teacher gets the paper values (Table 5.5)
7	The tokenization of the MMLU letters	Excluded: each letter is one token
8	The checkpoint selection	Excluded: the best step is the last evaluation in all runs but one
9	The version of the `transformers` library	Not tested

of the LoRA updates. At 10^{-3}, the learning rate of the paper, it discards every update of α, because α starts at 1.0.

A stage 1 run with the learning rate and the schedule of the paper showed this effect. In bf16, α stayed at exactly 1.000 in all 16 layers. The validation loss was 8.1641, 2.4 times the loss of the LoLCATs recipe. The same recipe with all weights, the optimizer states and the target in float32 moved α to 0.60–0.79 and lowered the loss to 4.9478. All later runs with the recipe of the paper use float32.

7.3 The recipe experiments

Each experiment changed one part of the recipe. These runs use feature dimension 32, the LoLCATs rule for a head dimension of 64. A separate run showed that this value is not a cause. It gave a loss of 3.4219 against 3.2549 for 128, and the gate saturated in the same way. Table 7.2 gives the results.

Each step toward the recipe of the paper lowered the loss. With all optimizer settings of Table 13, the recipe of the paper came near the LoLCATs recipe, but it did not pass it. The accuracies stayed in small ranges: 55.8–57.7 on PIQA and 34.1–36.5 on ARC-Easy. With unpaired SEs, none of these differences is clear.

One result was clear. The recipe of the paper stops the saturation of the gate. With the LoLCATs recipe, the gate stays above 0.999 for every token in 14 or 15 of the layers 1–15. With the recipe of the paper, no layer saturates fully. Thus the high learning rate of LoLCATs causes the saturation. But a gate that decays did not give a better

Table 7.2: Stage 1 runs with different recipes (project notes, documents 12 and 14). All runs use the original Lizard attention. Accuracy in % after stage 1. PIQA has an SE of approximately 1.2 points, and ARC-Easy approximately 1.0 point.

Stage 1 run	Loss	MMLU subset	PIQA	ARC-Easy
Feature dimension 128, LoLCATs recipe, bf16 (Run 1)	3.2549	22.5	57.6	39.0
Feature dimension 32, LoLCATs recipe, bf16	3.4219	23.2	–	–
Recipe of the paper, bf16	8.1641	25.3	55.8	34.1
Recipe of the paper, float32	4.9478	24.6	57.7	35.7
Config 1: also $\beta_2 = 0.99$ and a minimum learning rate	3.9764	26.7	57.3	36.5
Config 2: config 1 with gradient clipping at 1.0	3.5092	23.2	57.5	35.6

model either.

7.4 Stage 2 with the recipe of the paper

Stage 2 on config 1 used the stage 2 settings of Table 13 (learning rate 5×10^{-4}, LoRA on q, k and v) in float32. Its validation loss was 1.9158, 14.9% lower than in Run 2 (2.252). But its scores were almost the same: PIQA 67.7 against 67.95, and ARC-Easy 54.9 against 54.8. On the MMLU subset, it gave "A" for 77.5% of the questions. Figure 7.2 shows that both stage 2 recipes add approximately 10 points on PIQA and approximately 16–18 points on ARC-Easy. The stage 1 differences did not reach the final model. Stage 2 changed the memory of the gate: in layers 9–15, the weight kept after 512 tokens fell from 0.48–0.94 to below 0.003. The gate parameters stayed frozen. Thus LoRA changed the hidden states that the gate reads.

7.5 A lower loss does not give a higher accuracy

Over the stage 1 runs, the loss fell from 8.16 to 3.05, but PIQA and ARC-Easy did not increase. Over the six runs with ARC-Easy, the rank correlation (Spearman) between the loss and the accuracy is +0.31 for ARC-Easy and −0.06 for PIQA. A positive value means that a lower loss comes with a lower accuracy. With six runs, none of these values is clear. Figure 7.3 shows all stage 1 runs of this thesis. Only the runs with RoPE in the window (orange) and the LoLCATs control (green) leave the cluster.

Three properties of the loss can explain this:

- The loss measures each layer alone, with the input of the teacher. It cannot see an error that grows from layer to layer.
- All 2048 positions count the same. Positions 0–127 give only 6.25% of the loss. But almost all PIQA and ARC-Easy prompts are shorter than 128 tokens.

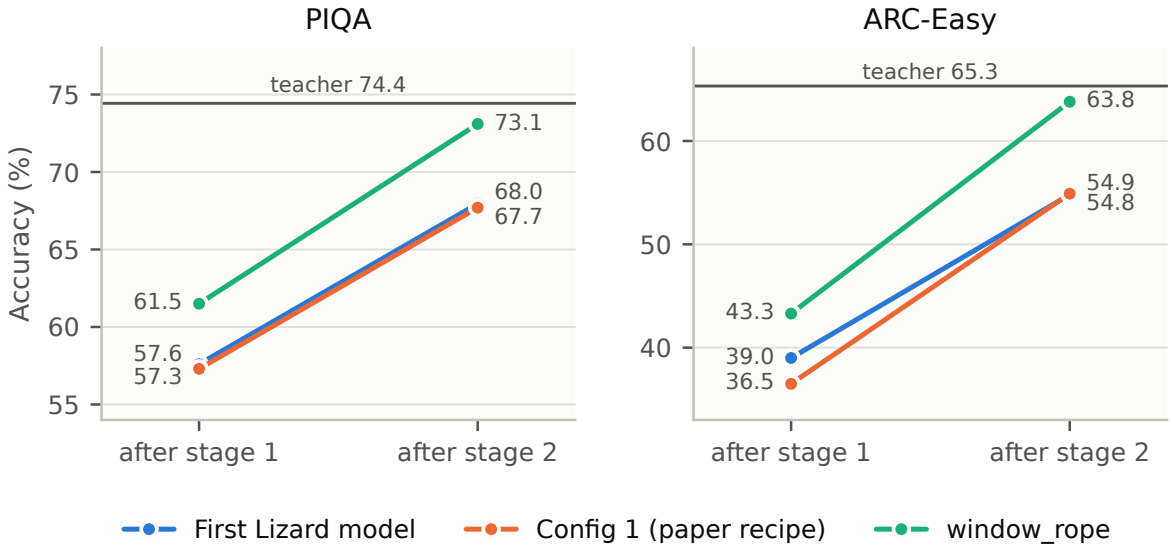


Figure 7.2: PIQA and ARC-Easy after stage 1 and after stage 2, for three models. The first Lizard model and config 1 end at almost the same values. The `window_rope` model of Chapter 9 starts higher and ends higher.

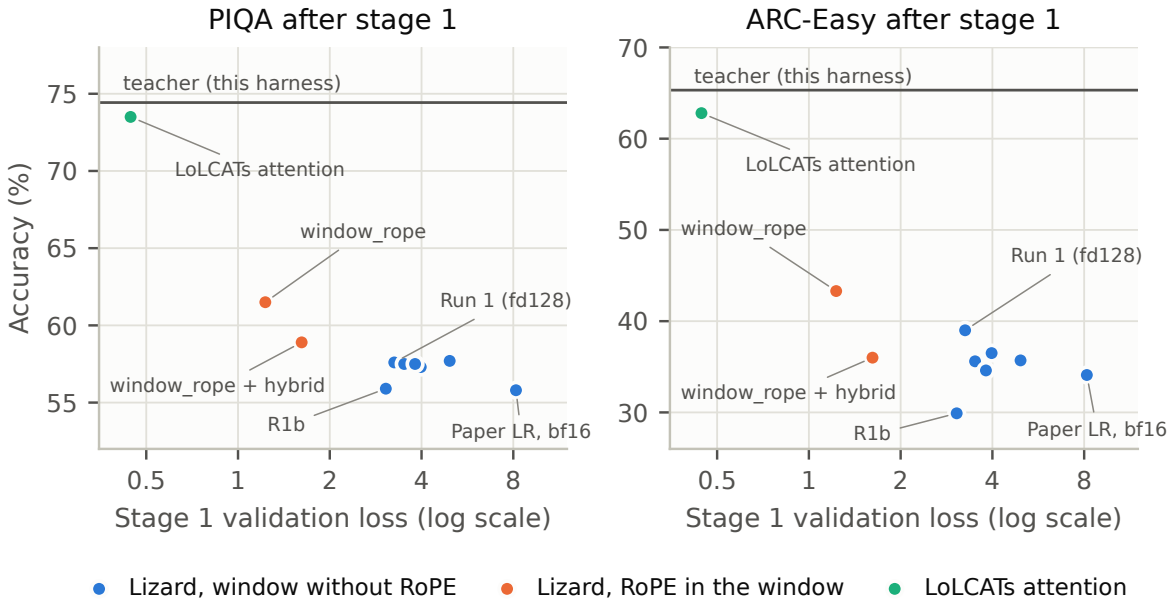


Figure 7.3: Stage 1 loss against the accuracy after stage 1. Each point is one stage 1 run. The unlabeled blue points are the paper recipe in float32, config 1, config 2 and C1. Values in Tables 7.2 to 7.4 and Appendix D.

- Each run has one seed. Thus the noise of the loss and of the scores is not known.

7.6 A limit for each head

The second gap analysis looked at the attention weights (Section 8.2) and found a limit in the architecture. A Lizard row is the gated branch, with weights that sum to 1, plus α times the window branch, with weights that sum to $\rho_i \leq 1$. Thus the share of the

exact window in a row has an upper limit:

$$\text{window share}_i = \frac{\alpha\rho_i}{1+\alpha\rho_i} \leq \frac{\alpha}{1+\alpha}. \tag{7.1}$$

All 32 heads of a layer share α, the gate, the feature maps and the sink logits. Only $\mathbf{q}$, $\mathbf{k}$ and $\mathbf{v}$ differ between the heads, and stage 1 does not train them. The measurements reach the limit. In layer 15, head 14 of config 1, $\alpha = 0.576$, so the limit is 0.365. The measured window share is 0.365. But the teacher puts 91.3% of the weight of this head inside the window. In layer 0, the LoLCATs recipe trained α down to 0.063. Then no head of layer 0 can take more than 6% from the window. But layer 0 has local heads, for example a head that attends to the previous token.

This limit explains several observations. Local teacher heads match worst. All heads of a layer get almost the same long-range share. The training lowers α for the majority of heads, which need far tokens, and the local heads lose. The gated branch cannot become local for one head only, because all heads share one gate. And without RoPE, neither branch knows the relative position of two tokens.

7.7 Design options of v2: C1, `joint` and `hybrid`

The options of v2 (Table 4.4) attack the limit. Table 7.3 gives three runs with the stage 1 recipe of config 1.

Table 7.3: Stage 1 runs with options of Lizard v2 (project notes, document 13). Accuracy in % after stage 1.

Run	Loss	PIQA	ARC-Easy	MMLU subset	"A"
Config 1 (reference)	3.9764	57.3	36.5	26.7	51.2%
C1: one α for each head	3.8092	57.5	34.6	23.9	64.9%
R1: per-head α and maps, `joint`	stopped	–	–	–	–
R1b: per-head α and maps, `hybrid`, α start 0.1	**3.0542**	55.9	**29.9**	24.9	90.2%

C1 lowered the loss by 4.2%, but the accuracy did not change. R1 stopped at 19% of the first epoch, because `joint` gives the gated branch almost no weight at the start values (Table 4.5, P6). R1b gave the lowest loss of all Lizard runs at that time, 23.2% below config 1. But ARC-Easy fell by 6.6 points ($z \approx -4.9$). In layer 15, the shared gate fell to 0.128. Heads 14 and 23 improved by 76–78%, but 28 of the 32 heads got worse. A short gate gives position information, so it helps local heads. But one gate for all heads takes the long range from the other heads.

7.8 The LoLCATs attention as a control

The control run used the same pipeline, data, recipe and harness, with the original LoLCATs attention in place of Lizard. Thus a clear difference must come from the attention layer. Table 7.4 gives the result.

Table 7.4: The LoLCATs control against Lizard after stage 1 (project notes, "LoLCATs control"). Both runs use the LoLCATs recipe.

Measure	Lizard, Run 1	LoLCATs control	Difference
Stage 1 loss	3.2549	**0.4442**	7.3× lower
PIQA	57.6	**73.5**	+15.9 ($z \approx 10$)
ARC-Easy	39.0	**62.8**	+23.8 ($z \approx 17$)
MMLU subset	22.5	26.0	+3.5 ($z \approx 1.0$)
"A" on MMLU	95.4%	19.3%	–
Probability mass of the four letters	0.018	0.962	–

After stage 1 alone, the LoLCATs control is above the final first Lizard model on PIQA (+5.5 points) and ARC-Easy (+8.0 points). Thus the pipeline can train a much better model, and the gap comes from the Lizard layer. On short prompts, the control computes almost exactly the teacher attention. Its window has RoPE and covers 128–255 tokens. Its linear part acts only on tokens farther back. Its layer-wise MSE is 3.7 to 36.9 times lower than that of Lizard in every layer. Layer 0 has the largest difference. There, the control gives its window 79% of the weight on far queries, but Lizard turned its window almost off ($\alpha = 0.042$). MMLU stays near chance for the control too, but the control keeps the answer format: 96.2% of its probability goes to the four letters.

7.9 Summary

RQ2. The recipe and the precision matter, but they do not explain the gap. bf16 discards small updates and freezes α at the learning rate of the paper. Lower stage 1 losses did not give higher accuracies. The evidence excludes the checkpoint loading, the temperature, the feature dimension and the harness. The LoLCATs control in the same pipeline reaches 98.8% of the teacher on PIQA after stage 1 alone. Thus the cause is in the Lizard attention. The most probable mechanism is a limit for each head. Its parts are the shared parameters, the gated branch with the weight 1, the ceiling $\alpha/(1+\alpha)$ and the window without RoPE.

CHAPTER 8

Analysis with XAI Methods

The scores of Chapters 6 and 7 show that the model is weak, but not why. This chapter addresses RQ4. It uses four methods that look inside the trained model. All four methods use saved checkpoints, so they need no new training. They ran on the A10 machine.

8.1 Layer-wise MSE

Method. The script `layer_mse.py` runs the stage 1 validation data (16 sequences, 32,768 tokens) through the teacher. Each Lizard layer gets the input of the teacher layer, as in stage 1. The script computes 1000× MSE for each layer and each head. The sum over the layers gives the stored stage 1 loss again: exactly for the float32 checkpoints, and within 0.1–0.2% for the bf16 checkpoints. The output of the teacher grows with the depth. Thus the script also gives the *relative* MSE, the MSE divided by the mean square of the teacher output.

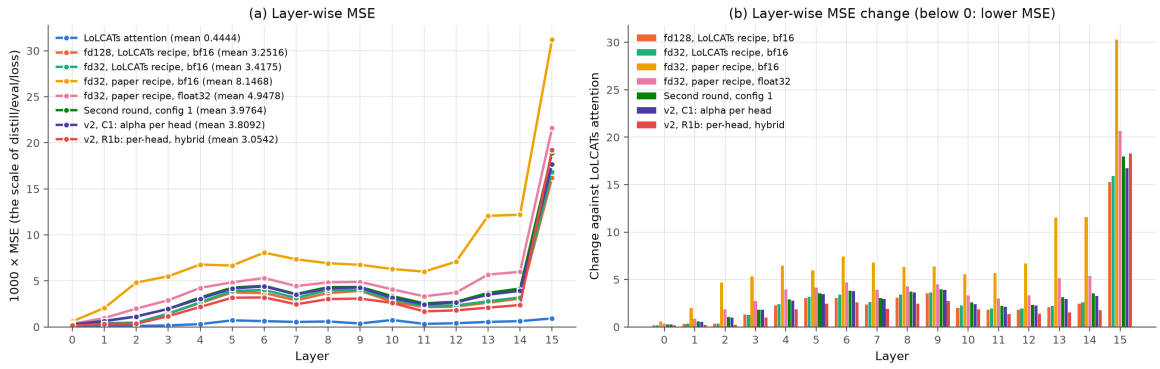


Figure 8.1: Layer-wise MSE of seven Lizard stage 1 runs and the LoLCATs control (blue line in panel a). Panel (b) shows the difference of each Lizard run to the control. The control has a lower error in every layer. Source: project notes, "XAI: layer-wise MSE".

Findings. Figure 8.1 shows the result.

- The MSE is small in the first layers and grows with the depth, as in Figure 14 of the LoLCATs paper. Layer 15 alone gives 24–31% of the stage 1 loss.
- Two heads of layer 15, heads 14 and 23, give approximately 20% of the loss in every Lizard checkpoint.
- Even with the best recipe, the error has 24–56% of the power of the teacher output in each layer. With the recipe of the paper in bf16, ten layers have a relative MSE above 1. There, an output of zero would be closer to the teacher.

- The recipes have the same order in all 16 layers. The difference between them is largest in the first and the last layers.
- The LoLCATs control has a lower MSE in every layer, against every Lizard checkpoint, by a factor of at least 3.0. Its error has only 1.1–11.0% of the power of the teacher output.
- Heads 14 and 23 of layer 15 are not hard for the control. They give 1.9% of its loss. Thus the teacher heads are not the problem. The Lizard layer is.
- Layer 0 has the largest ratio between Lizard and the control (30–121 times). This agrees with the control experiment: its window has RoPE and gives the local attention of layer 0.

8.2 Sample attention weights

Method. The script `attention_weights.py` computes the full weight matrix of each head for one sample of 1,024 tokens. For Lizard, the matrix is the gated weights plus α times the window weights. A check multiplies the weights with the values and gets the output of the layer again. The script compares each Lizard row with the teacher row. It uses the total variation (TV) distance after it divides the Lizard row by its sum. It also measures the weight outside the window and the weight on the first key. Plots like Figures 18–21 of the LoLCATs paper show the last 32 queries against the first 32 and the last 32 keys.

Findings.

- The trained checkpoints reproduce a large part of the teacher weights. Their mean TV distance is 0.31–0.33. The start weights have 0.71.
- The more local a teacher head is, the worse Lizard matches it. The correlation between the TV distance and the teacher weight outside the window is between -0.75 and -0.78. Heads with less than 30% of their weight outside the window have a mean TV distance of 0.56. Heads with more than 70% have 0.21–0.25.
- Lizard gives all heads of a layer almost the same long-range share. Inside a layer, the teacher share outside the window has a standard deviation of 0.126 over the heads. For Lizard, it is only 0.034–0.048.
- Layer 15, head 14 is a local head in the teacher. The teacher puts only 8.7% of its weight outside the window. Lizard puts 50–53% there. Its large MSE (Section 8.1) comes from too much weight far away in Lizard.
- Layer 0 is the worst layer (TV distance 0.49–0.60). Its head 2 attends to the previous token in the teacher. In all trained checkpoints, this head is almost empty (Figure 8.2). The output of layer 0 is small, so the MSE hardly shows this error.

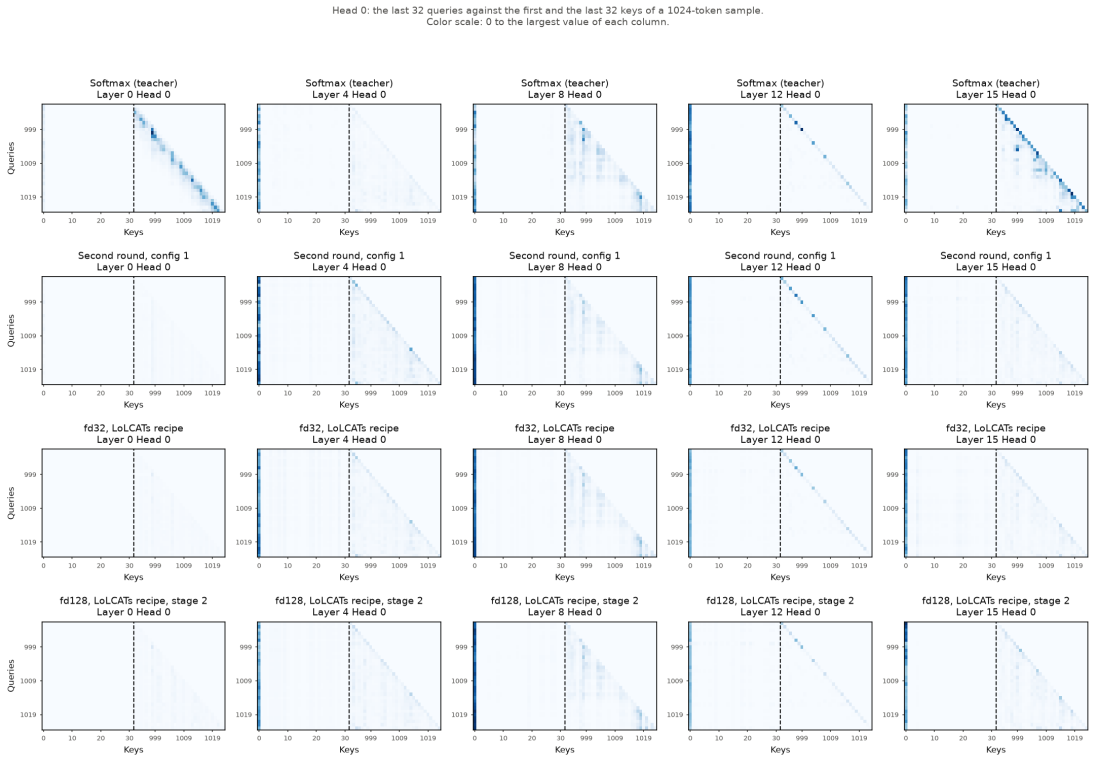


Figure 8.2: Attention weights of head 0 in five layers. The first row is the teacher. The other rows are three Lizard checkpoints. Each panel shows the last 32 queries against the first 32 keys (left of the dashed line) and the last 32 keys. The teacher has a sharp local pattern in layer 0. All Lizard checkpoints lose it. Source: project notes, "XAI: sample attention weights".

8.3 MSE by token position

Method. The script `position_mse.py` splits the query positions into three buckets: 0–127, 128–511 and 512–2047. In the first bucket, all keys are inside the window, as in almost every PIQA and ARC-Easy prompt. The script computes the relative MSE for each layer and bucket. A rank test then compares the error of each bucket with PIQA and ARC-Easy over the checkpoints.

Findings.

- The LoLCATs control computes the teacher attention at positions 0–127. Its relative MSE there is 8.4×10^{-6} to 4.4×10^{-5}, the size of bf16 rounding.
- Lizard misses 35–63% of the teacher output at the same positions (relative MSE 0.351–0.629 over 8 checkpoints). This is more than 20,000 times the value of the control.
- Layers 2 and 3 have the largest error at short positions. Layer 2 has the largest value in all 8 checkpoints (0.64–1.17).
- The error at positions 0–127 does not predict PIQA or ARC-Easy over the Lizard checkpoints. The rank correlations are not significant. Thus this error alone

cannot rank the stage 1 runs either.

The left panel of Figure 9.4 (Chapter 9) shows this measure for the first Lizard model.

8.4 Branch decomposition

Method. The script `branch_fit.py` asks which branch limits the fit. For each head and each bucket, it fits the teacher output $\mathbf{y}$ with a weighted sum of candidate outputs by least squares. It reports the remaining error divided by $\|\mathbf{y}\|^2$. The candidates are:

- G: the trained gated branch,
- W: the trained window branch, without RoPE and with the sinks,
- WR: an oracle window that uses the teacher scores with RoPE over the same 128 keys, without sinks and without trained parameters.

The script fits every combination, for example G+W or G+WR. It also reports the error of the trained output. At positions 0–127, WR is the teacher attention, so its error must be only rounding. This is the check of the method: WR gives 1.3×10^{-5} in bf16.

Table 8.1: Results of the branch decomposition for the eight Lizard stage 1 checkpoints without RoPE (project notes, "XAI: branch decomposition"). Values are remaining errors divided by $\|\mathbf{y}\|^2$.

Comparison	Result
W alone, positions 0–127, best weight for each head	0.394–0.458. The window without RoPE explains only approximately 60% of the teacher output.
G+W against W, positions 0–127	0.78–0.81 ×. The gated branch helps only a little inside the window.
G+WR against WR alone, positions 512–2047	0.40–0.58 ×. Outside the window, the gated branch gives a large part of the output.
G+WR against G+W, all positions	1.9–2.1 times less error. A window with RoPE would halve the error.
Heads 14 and 23 of layer 15, WR alone	0.017 and 0.134–0.136, against 0.57–0.62 and 0.90–0.94 for the trained output
Best weight of G in the G+W fit	Median 0.55–0.81, not 1. The gated branch has too much weight.

Table 8.1 gives the main results. The two heads with the largest error are window heads. With RoPE in the window, their error would almost disappear. The gated branch is not the main limit: outside the window, it carries the long range. But it has too much weight, because its weights always sum to 1 (Table 4.5, P8).

8.5 Why short prompts need a good window

The write-out files of the harness contain the prompt of each answer option. Table 8.2 gives the length of the longest option of each question.

Table 8.2: Prompt lengths of PIQA and ARC-Easy in characters (project notes, document 20). With approximately 4 characters for each token, the 95th percentile is approximately 88 tokens for PIQA and 77 tokens for ARC-Easy.

Task	Median	95th percentile	Maximum	Longer than 512 characters
PIQA	132	355	1,106	1.1%
ARC-Easy	139	311	802	0.3%

Thus for almost every question, all keys are inside the window of 128 tokens. On such a prompt, the teacher computes softmax attention with RoPE over all tokens. The LoLCATs control computes the same, up to rounding. Lizard computes the gated branch over all tokens plus α times the window without RoPE over the same tokens. Thus the window of Lizard must do the work of the teacher, but without RoPE it cannot give the same weights. The gated branch adds weight to tokens that the window already counts.

8.6 What the XAI results show together

> **RQ4, first answer.** The four methods agree. The largest errors are in local heads: head 2 of layer 0 and heads 14 and 23 of layer 15. These heads need exact attention to nearby positions. The window branch without RoPE cannot give it, and the shared gate and α cannot compensate for one head only. The gated branch carries the long range, but it has too much weight at short positions. Ranking for PIQA and ARC-Easy: the window without RoPE is the most probable cause, the gated branch is second, and the Hedgehog maps are third. The direct test is a window with RoPE (Chapter 9).

One method did not work as expected: the error at short positions did not predict the accuracy over the Lizard checkpoints (Section 8.3). Thus the XAI methods show the mechanism, but they do not replace the evaluation of each model.

CHAPTER 9

RoPE in the Window Branch

This chapter addresses RQ3. It tests the cause that Chapter 8 found: the window branch without RoPE. The change is one option of v2, `window_rope`. All other settings are the settings of Run 1: feature dimension 128, the LoLCATs recipe and bf16. The paper of Lizard writes the window without RoPE. Thus this model is an extension of Lizard, not the reproduction.

9.1 Why RoPE in the window, and the predictions

The branch decomposition gave reference values for Run 1. A conversion of the remaining errors into stage 1 losses gave estimates before the run. The trained output of Run 1 converts to 3.25. The best mix of the two trained branches (G+W) gives 2.96. A window with RoPE alone (WR) gives 2.43. The trained gated branch with a window with RoPE (G+WR) gives 1.20. Table 9.1 lists the six predictions and the results.

Table 9.1: Predictions before the `window_rope` run and the results (project notes, "RoPE in the window branch"). All six predictions hold.

Prediction	Result
1. The stage 1 loss falls clearly below 3.25, probably below 2.96.	1.2290. It is also below 2.43, so the trained gated branch adds to the window with RoPE.
2. The loss does not reach 1.20, because the gated branch keeps the weight 1.	1.2290, 2% above 1.20
3. The relative MSE at positions 0–127 falls clearly from 0.463, but stays above 0.	0.321 (−31%)
4. Layers 0 and 15, and heads 14 and 23 of layer 15, improve most.	Layer 15: 5.3×, layer 0: 3.7×. Heads 14 and 23: 13.7× and 11.3×.
5. α of layer 0 rises above 0.042.	0.660
6. PIQA and ARC-Easy increase, PIQA by at least 3.2 points.	PIQA +3.9, ARC-Easy +4.3

9.2 What RoPE in the window means for the linearization

RoPE is only in the window branch. The function `window_inputs` rotates the queries and keys of the window. The function `feature_maps` gets the queries and keys without RoPE. Thus the kernel $\phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_t)$ of the gated branch depends only on the content of the tokens.

The window branch is not a linear attention. It is a softmax attention over the last 128 keys and the 4 sinks. Linearization applies only to the gated branch. Thus RoPE in the window does not change the cost or the memory. Each key gets its rotation one time, at its own position, when it goes into the cache. A RoPE score depends only on the distance (Equation (2.2)), so the rotated keys in the cache stay valid (Table 4.5, P5).

Inside the window, the scores are the scores of the teacher. With RoPE, the window score is $(R_i\mathbf{q}_i)^\top(R_t\mathbf{k}_t)/\sqrt{d}$, as in the teacher. Thus for the keys inside the window, the window weights are proportional to the weights of the teacher. Only the denominator differs: it is the sum over the window and the sinks, not over all earlier keys. Without RoPE, the window knows which keys are inside it, but not their order.

The two branches get separate tasks. The window gives the local attention of the teacher. The gated branch must only approximate the part of the teacher output from keys outside the window. Thus the error of the linearization moves to the long range.

The gated branch still has no position code. Its only position information is the decay $\prod\gamma$. In `window_rope`, 99.9–100.0% of the tokens in layers 1–15 have $\gamma > 0.999$. Figure 9.1 shows what this means. If all gates are above 0.999, a token keeps more than $0.999^{128} \approx 0.88$ of its weight after 128 tokens and more than $0.999^{2048} \approx 0.13$ after 2048 tokens. Thus in these layers, the gated branch is almost a mean of the earlier tokens, with weights from the feature similarity. Layer 0 is different: its gate keeps 0.87 of the weight after 128 tokens.

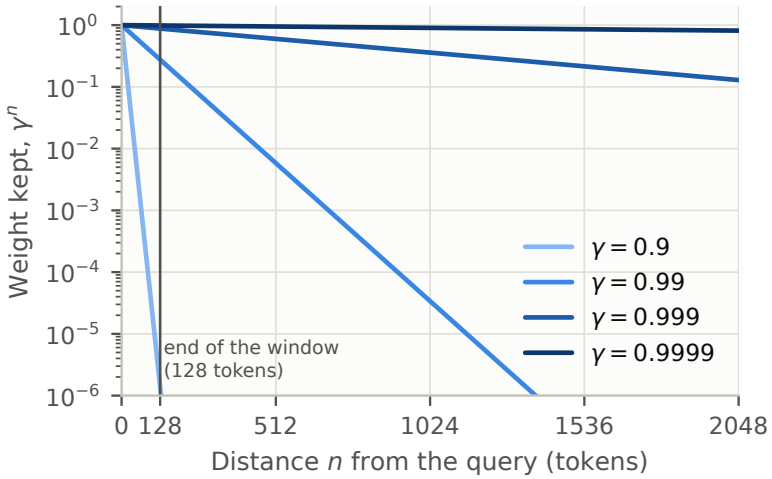


Figure 9.1: Weight γ^n that a constant gate keeps after n tokens. In `window_rope`, almost all gates of layers 1–15 are above 0.999. Then the gated branch hardly prefers recent tokens.

Table 9.2 compares the methods. The gated branch could also get RoPE, as in LoLCATs: $\phi(R_i\mathbf{q}_i)^\top\phi(R_t\mathbf{k}_t)$. The form stays linear and recurrent. But the Hedgehog map is not linear, so this kernel is not a function of the distance only. The code does

not have this option yet.

Table 9.2: Where each method uses RoPE (project notes, document 15 and "RoPE in the window branch").

Method	Window branch	Linear or gated branch
LoLCATs	RoPE	RoPE: the feature maps get the queries and keys after RoPE
Lizard paper	No RoPE	No RoPE. Position only from the gate.
Liger	Paper: no RoPE. Code: RoPE.	No RoPE
`window_rope`	RoPE	No RoPE. Position only from the gate.

9.3 Results after stage 1

Table 9.3 gives the scores after stage 1. PIQA and ARC-Easy rise clearly. These are the highest scores of all Lizard stage 1 checkpoints. The loss falls by 62% and closes 72% of the gap to the LoLCATs control. But PIQA closes only 25% of its gap, and ARC-Easy 18%. The collapse to "A" on MMLU is mostly gone.

Table 9.3: `window_rope` after stage 1 against Run 1 and the LoLCATs control (project notes, "RoPE in the window branch"). z values with unpaired SEs.

Measure	Run 1	`window_rope`	Difference	LoLCATs control
Stage 1 loss	3.2549	**1.2290**	−62%	0.4442
PIQA	57.6 ± 1.2	**61.5** ± 1.1	+3.9 ($z \approx 2.4$)	73.5
ARC-Easy	39.0 ± 1.0	**43.3** ± 1.0	+4.3 ($z \approx 3.0$)	62.8
MMLU subset	22.5	24.6	+2.1 ($z \approx 0.6$)	26.0
"A" on MMLU	95.4%	40.0%	–	19.3%
α of layer 0	0.042	0.660	–	–

Figure 9.2 shows that the MSE falls in every layer, most of all in layers 15 and 0. Heads 14 and 23 of layer 15 now give 4.4% of the loss, against 21% in Run 1. For the first time, a Lizard checkpoint has less than 3.0 times the MSE of the control in some layers (10 of 16).

The branch decomposition explains why the accuracy rose less than the loss (Figure 9.3). At positions 0–127, the window with RoPE alone now keeps an error of only 0.048 (Run 1: 0.403). The rest comes from the sinks. But the trained output has an error of 0.321 there, while the best mix of the same two branches has 0.025. Thus at short positions, the *mix* of the branches gives most of the error. The best weight of the gated branch is 0.08 at positions 0–127, 0.65 at positions 128–511 and 0.79 at positions 512–2047. With `gla_norm: row`, the gated branch keeps the weight 1 at

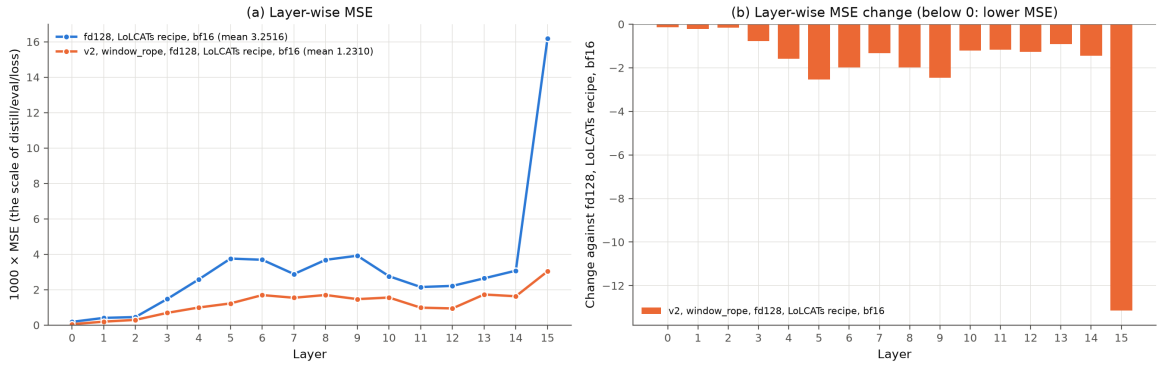


Figure 9.2: Layer-wise MSE of `window_rope` (orange) against Run 1 (blue). Panel (b) shows the difference. Source: project notes, "RoPE in the window branch".

every position. In the stage 1 loss, 94% of the positions are above 127. Thus the training sets the mix for long contexts, and the short prompts of PIQA and ARC-Easy get a mix that does not fit them.

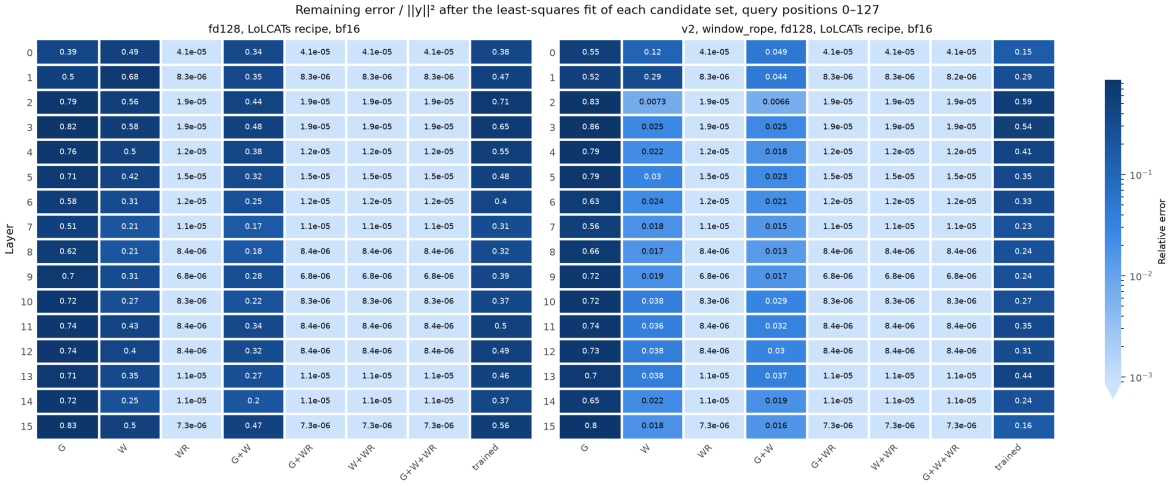


Figure 9.3: Branch decomposition at positions 0–127 for Run 1 (left) and `window_rope` (right). Each cell is the remaining error after the best fit of a set of branches. In `window_rope`, the window W alone is almost exact (light cells), but the trained output (last column) is not. Source: project notes, "RoPE in the window branch".

9.4 Results after stage 2

Stage 2 trained only the LoRA weights, as in Run 2. Its validation loss fell from 1.3432 to 1.2352, against 2.252 for Run 2. Table 9.4 gives the scores.

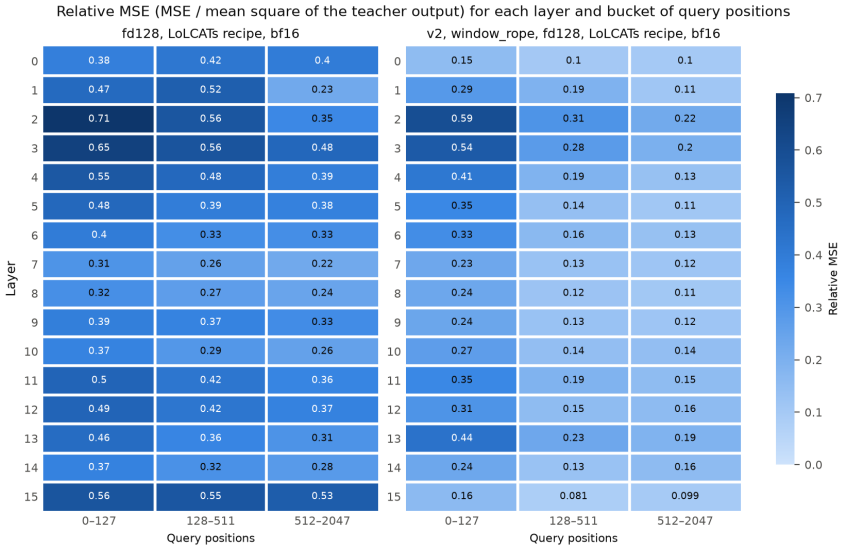


Figure 9.4: Relative MSE by layer and position bucket for Run 1 (left) and `window_rope` (right). Short positions now have the largest error, most of all in layers 2 and 3. Source: project notes, "RoPE in the window branch".

Table 9.4: `window_rope` after stage 2 (project notes, "RoPE in the window branch", Run 2, and document 7, section 6). Accuracy in %.

Task	First Lizard model	`window_rope`	Difference	Teacher, this harness
PIQA	67.95 ± 1.09	**73.1** ± 1.0	+5.1 ($z \approx 3.4$)	74.4 ± 1.0
ARC-Easy	54.8 ± 1.0	**63.8** ± 1.0	+9.0 ($z \approx 6.3$)	65.3 ± 1.0
MMLU subset	24.9	25.6	+0.7	33.7
MMLU, all questions	23.3	**27.2** ± 0.4	+3.9	31.7 ± 0.4
Stage 2 validation loss	2.252	1.235	–	–

After stage 2, `window_rope` is the best Lizard model of this work. It gets 98.2% of the teacher on PIQA and 97.7% on ARC-Easy (Figure 9.5). The differences to the teacher are −1.4 points ($z \approx -0.9$) and −1.5 points ($z \approx -1.1$), so they are not clear. The gain of stage 1 grows in stage 2: from +3.9 to +5.1 points on PIQA, and from +4.3 to +9.0 points on ARC-Easy.

9.5 The shared denominator does not help

The mix problem of Section 9.3 suggested the next step: the shared denominator of LoLCATs (`gla_norm: hybrid`). With this option, the share of the gated branch can grow with the position. The run started α at 0.1, as LoLCATs does. Table 9.5 gives the result.

The gate split the layers into two groups. In 7 layers, the gate stayed open, and the MSE fell at all positions. In layer 0, the MSE fell 5.8 times. In the other 9 layers, the gate closed: γ fell below 0.001 for almost all tokens, so the gated branch kept only the

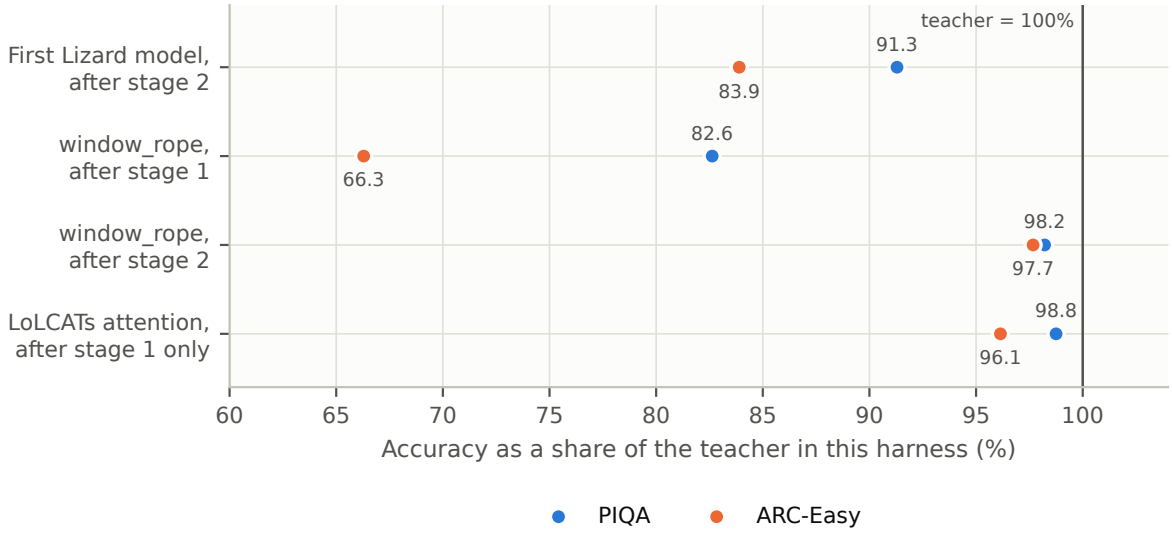


Figure 9.5: PIQA and ARC-Easy as a share of the teacher in this harness (74.43 and 65.32). After stage 2, `window_rope` reaches 98.2% and 97.7%. The LoLCATs control reaches a similar level after stage 1 alone.

Table 9.5: `window_rope` with and without `hybrid`, after stage 1 (project notes, "`window_rope` with the shared denominator").

Measure	`window_rope`	with `hybrid`	Difference
Stage 1 loss	**1.2290**	1.6172	+32%
PIQA	**61.5**	58.9	-2.6 ($z \approx -1.6$)
ARC-Easy	**43.3**	36.0	-7.3 ($z \approx -5.1$)
MMLU subset	24.6	25.3	+0.7
Relative MSE, positions 0–127	0.321	**0.222**	−31%

current token. These 9 layers lost the long range. Their MSE was 1.8–2.7 times that of `window_rope`, and they gave 78% of the loss. The error at short positions fell, but PIQA and ARC-Easy fell too. Thus `window_rope` with `row` stayed the best setting.

9.6 Table 9 of the paper

Table 9.6 compares `window_rope` with Table 9 of the paper. The paper used a newer harness, so only the teacher rows of the same harness give the real gap. ARC-Challenge ran on 2026-10-11, after the last update of the project notes. HellaSwag and WinoGrande were still running at the time of this draft.

On ARC-Challenge, `window_rope` gets 36.1 with `acc_norm` (33.2 with `acc`). This is within 0.4 points of the paper values for the teacher and for Lizard. The teacher of this harness has no ARC-Challenge value yet.

Table 9.6: `window_rope` after stage 2 against Table 9 of the paper. Accuracy in %. ARC-Challenge, HellaSwag and WinoGrande use `acc_norm`, `acc_norm` and `acc`, as in the paper.

Model	PIQA	ARC-e	ARC-c	Hella-Swag	Wino-Grande	MMLU
Teacher, this harness	74.4	65.3	–	–	–	31.7
`window_rope`, this harness	73.1	63.8	36.1	pending	pending	27.2
Teacher, paper	74.1	65.4	36.4	63.8	60.0	31.0
LoLCATs, paper	74.6	63.0	35.1	63.7	61.5	27.3
Lizard, paper	74.8	65.6	36.5	64.1	59.7	29.8

9.7 MMLU on all questions

The MMLU subset has an SE of 2.6 points. Thus both models also ran on all 14,042 questions (Table 9.7).

Table 9.7: MMLU on all questions, 5-shot (project notes, "RoPE in the window branch", findings 15–21).

Measure	Teacher	`window_rope`
Mean over the subjects	**31.7** ± 0.4	**27.2** ± 0.4
Mean over the questions	31.0	26.9
Probability mass of the four letters	0.999	0.975
Confidence (largest letter probability)	0.342	0.354
Entropy over the four letters	1.904 bits	1.900 bits

The model is above chance, but it keeps only one third of the knowledge. 27.2 is approximately 5.9 SE above 25%. The first Lizard model was 4.8 SE below. On the same questions, the teacher has 4.2 more points (paired SE 0.44, $z \approx 9.5$). The teacher alone is right on 2,210 questions, and the student alone on 1,625 (McNemar $z \approx 9.5$). The student gets 86% of the teacher score, but only 33% of the part of the teacher score above chance: $(27.2 - 25)/(31.7 - 25)$.

The student follows the teacher. When the teacher is right, the student is right on 49.3% of the questions. When the teacher is wrong, the student is right on only 16.8%. Without a link between the models, both values would be 26.9%.

The student loses most where the teacher knows most. Figure 9.6 shows the four subject groups. The gap is 2.8 and 2.7 points in STEM and the humanities, where the teacher is near chance. It is 4.7 and 7.2 points in the social sciences and the group "other", where the teacher knows more. Figure 9.7 shows the same pattern for the single subjects.

The gap does not grow with the prompt length. In the four quarters of the prompt length, the gap is 5.9, 3.8, 2.4 and 4.5 points. The shortest prompts have the

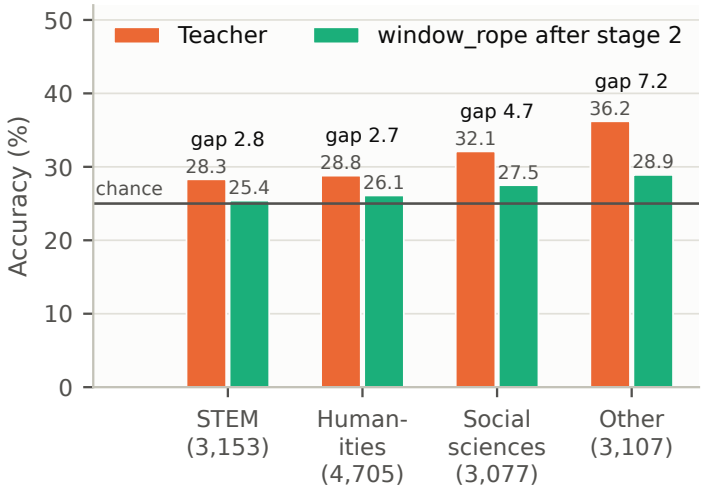


Figure 9.6: MMLU by subject group, with the number of questions. The values are means over the questions of each group.

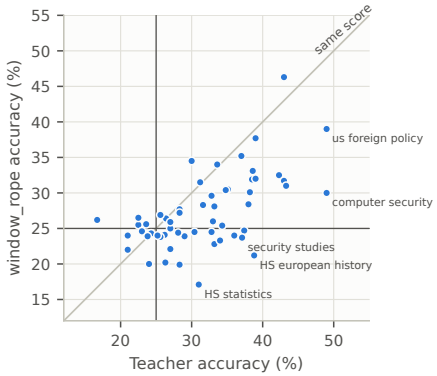


Figure 9.7: MMLU for each of the 57 subjects. Points below the diagonal are subjects where the student loses. Data in Appendix E.

largest gap. The length and the subject are not independent, but this result does not support the idea that the long 5-shot context causes the gap.

9.8 Letters and ties on MMLU

Figure 9.8 shows the letters that the two models select. Both models prefer "A" and almost never give "D". Thus the even spread over the letters is a property of the 1B teacher in this format, not a sign of damage in the student. The gap is in the choice of the right letter.

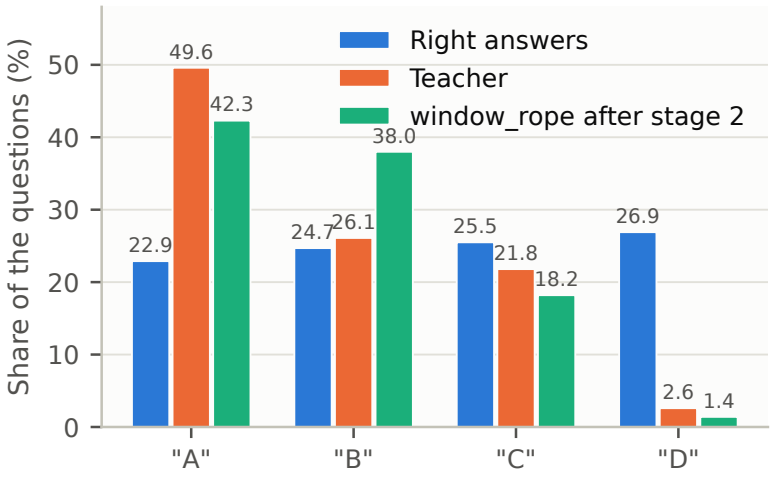


Figure 9.8: Selected letters on all MMLU questions, and the share of each letter among the right answers.

The letter log-likelihoods of both models are multiples of 0.125 or 0.25, because the logits are in bf16. Thus ties at the maximum are common: 24.2% of the teacher answers and 20.5% of the student answers. In a tie, the harness selects the first letter. Suppose that a random choice breaks each tie. Then the mean over the questions is 31.7 for the teacher (stored 31.0) and 27.0 for the student (stored 26.9). The ties add to the preference for "A", but the rare "D" does not come from the ties.

RQ3. RoPE in the window branch closes almost all of the gap on PIQA and ARC-Easy. It does not close the gap on MMLU: 27.2 against 31.7 for the teacher. The shared denominator does not help, because the gate closes in 9 of 16 layers.

CHAPTER 10

Discussion

10.1 Answers to the research questions

RQ1 **Can the Lizard results for Llama-3.2-1B be reproduced?** Not with the method as written. The first Lizard model got 23.3 on MMLU, 68.0 on PIQA and 54.8 on ARC-Easy, against 29.8, 74.8 and 65.6 in the paper (Chapter 6). The extension `window_rope` comes close on PIQA and ARC-Easy, but it is not the method of the paper.

RQ2 **Which differences explain the gap?** Not the recipe alone and not the code. The tests found no bug. The recipe of the paper in float32 did not give a better model than the LoLCATs recipe. The LoLCATs control in the same pipeline reached the teacher on short tasks after stage 1. Thus the attention layer causes most of the gap (Chapter 7).

RQ3 **Which changes close the gap?** RoPE in the window branch closes most of the gap on PIQA and ARC-Easy: 98.2% and 97.7% of the teacher. One α for each head, feature maps for each head and the shared denominator did not help (Chapters 7 and 9).

RQ4 **Do the measurements inside the model explain the scores?** Partly. They found the mechanism: local heads, a window without RoPE and a gated branch with too much weight at short positions. They predicted the effect of RoPE in the window correctly. But the error at short positions did not rank the checkpoints by accuracy (Chapter 8).

10.2 Why PIQA and ARC-Easy recover, and MMLU does not

PIQA and ARC-Easy prompts are short, so the window does almost all the work. A window with RoPE computes the teacher scores inside the window. Thus these two tasks mostly test the window, and `window_rope` reaches the teacher. MMLU is different in three ways. The 5-shot prompts have up to 2048 tokens, so the gated branch acts on most of the context. The task tests knowledge, and the teacher itself is only 6.7 points above chance. And the answer is one letter, so the model must connect the right option with its letter. Bick et al. [3] explain that a few sharp retrieval heads do this step, and that recurrent models give smoother attention. This agrees with the results, but the project did not test it directly.

The full MMLU results add two facts. First, the student follows the teacher: it is right much more often when the teacher is right. Thus the student keeps part of the knowledge. Second, the gap does not grow with the prompt length. Thus the long context alone does not explain the gap. The stage 2 data may also matter: stage 2 uses the next-token loss on Alpaca, not the logits of the teacher.

10.3 Differences from the paper that stay

Some differences from the paper remain. The paper has no public code. The open points D1, D2, D3 and D5 (Table 4.3) allow other readings. The evaluation harness is older than the harness of the paper, although the teacher scores agree within 0.7 points. A web search found an ACL 2026 version of the paper. Its abstract reports MMLU gains of 9.4–24.5 points, but this work did not read its tables. The Liger code applies RoPE in the window although the Liger paper does not write it. Thus the code of the Lizard authors may also differ from their paper. This is a possibility, not a fact. A message to the authors is the direct way to find out.

10.4 Limits of this study

- **One seed.** Each run has one random seed. Thus the variance from run to run is not known, and small differences between runs can be noise.
- **Small question sets.** Many early decisions used the MMLU subset, with an SE of 2.6 points. Only the final models ran on all questions.
- **bf16 ties.** The bf16 logits give ties in 20–24% of the MMLU answers, and the harness then selects the first letter. This changes the accuracy by up to 0.7 points.
- **Unpaired statistics.** Most comparisons use unpaired SEs. Paired tests would be more sensitive.
- **No efficiency measurement.** The code is a dense reference that materializes $L \times L$ matrices. This thesis does not measure the speed or the memory of the trained model on long inputs.
- **Unchecked sources.** Some related work comes from web searches and from ChatGPT answers. The full texts of these sources were not read.

10.5 Lessons for reproduction work

1. **Test the code first.** The tests against a slow reference excluded a whole class of causes in a short time.
2. **Change one thing at a time.** Each experiment changed one setting. Thus each result has a clear cause.

3. **Measure the accuracy, not only the loss.** The stage 1 loss fell from 8.16 to 3.05 over the runs, but the accuracy did not rise.
4. **Use a control.** The LoLCATs control showed in one run that the pipeline works and that the attention layer is the cause.
5. **Read the code of related methods, not only their papers.** The RoPE in the window of the Liger code was an important hint.

CHAPTER 11

Conclusion and Future Work

11.1 Summary

This thesis reproduced the Lizard linearization on Llama-3.2-1B in the open LoLCATs code base. The first model stayed far below the paper, and on MMLU it gave "A" for almost every question. Tests excluded a bug in the formulas. Experiments with the recipe, the precision and the readings of the paper lowered the stage 1 loss, but they did not raise the accuracy. A control with the LoLCATs attention showed that the cause is in the Lizard layer. Four interpretability methods found the most probable mechanism: the window branch without RoPE cannot reproduce the local heads of the teacher. With RoPE in the window, the model reaches 98.2% of the teacher on PIQA and 97.7% on ARC-Easy. On MMLU, it gets 27.2 against 31.7 for the teacher.

11.2 Future work

1. **Complete Table 9.** Evaluate HellaSwag and WinoGrande for `window_rope`, and ARC-Challenge, HellaSwag and WinoGrande for the teacher in the same harness.
2. **Checks on MMLU without training.** Evaluate MMLU 0-shot, with a beginning-of-text token, with rotated options and without the gated branch. These checks separate a loss of knowledge from a failure of the letter step.
3. **Softmax layers or heads.** Keep one or two layers as softmax, selected by the KL divergence or by a search for G&A heads [3, 17]. LoLCATs and Lizard report large MMLU gains with such hybrids at 8B.
4. **A KL loss in stage 2.** Train stage 2 on the logits of the teacher, as RADLADS [9] does.
5. **Less gated weight at short positions.** Give the gated branch only the keys outside the window. Another way is to start the gate near 0.95 with the shared denominator, so that the gates do not close.
6. **RoPE in the gated branch.** Test $\phi(R_i\mathbf{q}_i)^\top\phi(R_t\mathbf{k}_t)$ as a new option.
7. **More seeds and an efficiency benchmark.** Repeat the best run with other seeds. Measure the speed and the memory for long inputs with a chunked kernel.
8. **A hyperparameter search.** The project notes plan an Optuna study with the mean of PIQA and ARC-Easy as its score.

APPENDIX A

Definitions

This appendix gives short definitions of terms that the thesis uses.

Causal language modeling
: Training a model to predict the next token from the earlier tokens. The loss is the cross-entropy of the right next token. Stage 2 uses this loss.

Fine-tuning
: More training of a trained model on new data, often with a small learning rate or with few trainable parameters.

Token
: A piece of text, for example a word or a part of a word. The model reads and writes tokens.

Context size
: The number of tokens that the model reads at one time. The training of this thesis uses 2048 tokens.

Packing
: Joining many short examples into one long sequence, so that no space is lost. Each training sequence of this thesis has exactly 2048 tokens from several Alpaca examples.

EOS token
: A special token that marks the end of a text. With packing, it separates the examples in a sequence.

BOS token
: A special token at the beginning of a text. The harness of this thesis does not add it for Llama.

Padding
: Extra tokens that make short sequences as long as the longest sequence of a batch. The model must ignore them.

Batch and gradient accumulation
: A batch is the set of sequences for one gradient step. With gradient accumulation, the trainer adds the gradients of several small batches before it makes one step. This thesis uses 1 sequence and 8 accumulation steps.

Epoch
: One pass over all training data.

Learning rate schedule
: The rule that changes the learning rate during training. *Warmup* increases it from a

small value at the start. A *cosine* schedule then decreases it along a cosine curve. *Reduce on plateau* decreases it only when the validation loss stops improving.

Gradient clipping

A limit on the norm of the gradient. A larger gradient gets a smaller scale before the step.

bf16 and float32

Two number formats. float32 has 24 bits of precision. bf16 has 8 bits of precision, but the same range as float32. bf16 needs half the memory. But it cannot store small changes to a weight.

Log-likelihood

The logarithm of the probability that the model gives to a text. The harness uses it to compare answer options.

Few-shot

A prompt with some solved examples before the question. MMLU in this thesis uses 5 examples (5-shot).

Standard error

The expected spread of a measured value around its true value. For an accuracy p on n questions, it is $\sqrt{p(1-p)/n}$.

APPENDIX B

Formulas and Proof Sketches

This appendix gives short derivations for the main claims of Table 4.5. The project notes (documents 13, 14 and 15) give the full audits and the float64 checks.

B.1 The parallel form equals the recurrent form (P1)

Let $\mathbf{S}_0 = 0$ and $\mathbf{S}_i = \gamma_i \mathbf{S}_{i-1} + \phi_k(\mathbf{k}_i)\mathbf{v}_i^\top$. By induction over i,

$$\mathbf{S}_i = \sum_{t=1}^{i} \left(\prod_{l=t+1}^{i} \gamma_l \right) \phi_k(\mathbf{k}_t)\, \mathbf{v}_t^\top. \tag{B.1}$$

The step from $i-1$ to i multiplies every term by γ_i and adds the term $t = i$, with an empty product equal to 1. Thus $\phi_q(\mathbf{q}_i)^\top \mathbf{S}_i$ is the numerator of Equation (2.6). A second state $\mathbf{z}_i = \gamma_i \mathbf{z}_{i-1} + \phi_k(\mathbf{k}_i)$ gives the denominator in the same way. The denominator is at least $\phi_q(\mathbf{q}_i)^\top \phi_k(\mathbf{k}_i) > 0$, because all features are positive. The float64 check gave a difference of 2.2×10^{-16}.

B.2 The total weight of a row and the window share (P2, P8)

The gated weights of row i sum to 1, because the branch divides by its own row sum. The window weights sum to

$$\rho_i = \frac{\sum_{t \in W_i} e^{s_{it}}}{\sum_j e^{t_j} + \sum_{t \in W_i} e^{s_{it}}} \leq 1. \tag{B.2}$$

Thus the total weight of the row is $1 + \alpha\rho_i$. For $\alpha \geq 0$, the share of the window is $\alpha\rho_i/(1 + \alpha\rho_i)$. This value grows with $\alpha\rho_i$, so it is at most $\alpha/(1+\alpha)$. With one α for each head (C1), the limit becomes $\alpha_h/(1+\alpha_h)$ for each head. Without the denominator in the gated branch, the limit disappears.

B.3 Four equal sinks act as one sink (P7)

The sinks enter the loss only through $\sum_j \exp(t_j)$. Thus equal logits get equal gradients. Adam updates each element from its own history of gradients. Equal histories give equal updates, so by induction the logits stay equal. Then $\sum_{j=1}^{4} e^t = e^{t+\ln 4}$, which is

one sink with the logit $t + \ln 4$. All runs of this thesis show four equal sink logits in each layer.

B.4 The written sink term can break the denominator (P4)

The paper writes $\sum_j t_j$ in the denominator. With $m = 4$ sinks, one key with the score 0 and $t_j = -0.25$ for all j, the denominator is $4 \cdot (-0.25) + e^0 = 0$. With $t_j = -1$, it is -3. With $\sum_j \exp(t_j)$, the denominator is always positive.

B.5 The log-space form and the softmax map (P3)

The log-space form adds $\log c_t$ to the argument of the feature map. For the exp map, $\exp(\mathbf{x}W + \log c_t) = c_t \exp(\mathbf{x}W)$, which gives the gate product. For the softmax map, the same constant appears in every element of the softmax and cancels: $\mathrm{softmax}(\mathbf{x}W + \log c_t) = \mathrm{softmax}(\mathbf{x}W)$. Thus with the softmax map of Table 13, the gate disappears from this form.

B.6 The shared denominator (`hybrid`)

Let $w_{it} = c_{it}\, \phi_q(\mathbf{q}_i)^\top \phi_k(\mathbf{k}_t)$ be the gated weights and m_i the larger of the largest window score and the largest sink logit. The option `hybrid` computes

$$\mathbf{y}_i = \frac{\sum_{t \le i} w_{it}\, \mathbf{v}_t + |\alpha| \sum_{t \in W_i} e^{s_{it} - m_i}\, \mathbf{v}_t}{\sum_{t \le i} w_{it} + |\alpha| \left(\sum_{t \in W_i} e^{s_{it} - m_i} + \sum_j e^{t_j - m_i} \right)}. \tag{B.3}$$

The row is a weighted mean of the values with a total weight below 1. The share of the gated branch is its part of the denominator. If the gates are 1, the gated sum grows with the position i, but the window sum has at most 128 terms. Thus the window share falls approximately as $1/i$ (P9).

APPENDIX C

Configurations of the Runs

Each run uses three config files of the LoLCATs code base. The model config is in `configs/model`. The distill config for stage 1 and the finetune config for stage 2 are in `configs/experiment`. Table C.1 lists them.

Table C.1: Config files of the runs. All model configs start with `distill_llama3_2_1b_lizard`, which the table writes as `...`.

Run	Model config	Stage 1 / stage 2 config
Run 1 and Run 2	`..._w128_fd128_m4`	`..._lr1e-2_1b` / `finetune_lora_qkvo_alpaca_clean_1b`
Feature dimension 32	`..._w128_fd32_m4`	`..._lr1e-2_1b`
Paper LR, bf16	`..._w128_fd32_m4`	`..._lr1e-3_cosine_1b`
Paper LR, float32	`..._w128_fd32_m4_fp32`	`..._lr1e-3_cosine_1b`
Config 1	`..._w128_fd32_m4_fp32`	`..._lr1e-3_paper_noclip_1b` / `finetune_lora_qkv_alpaca_clean_paper_noclip_1b`
Config 2	`..._w128_fd32_m4_fp32`	`..._lr1e-3_paper_1b`
C1	`..._v2_w128_fd32_m4_fp32_alphahead`	`..._lr1e-3_paper_noclip_1b`
R1	`..._v2_w128_fd32_m4_fp32_perhead_joint`	`..._lr1e-3_paper_noclip_1b`
R1b	`..._v2_w128_fd32_m4_fp32_perhead_hybrid`	`..._lr1e-3_paper_noclip_1b`
`window_rope`	`..._v2_w128_fd128_m4_windowrope`	`..._lr1e-2_1b` / `finetune_lora_qkvo_alpaca_clean_1b`
`window_rope` + `hybrid`	`..._v2_w128_fd128_m4_windowrope_hybrid`	`..._lr1e-2_1b`
LoLCATs control	`distill_llama3_1_1b_lk_smd_wtk64_fd64_w01`	`..._lr1e-2_1b`

The distill configs start with `distill_alpaca_clean_xent0_mse1000`. The table writes this part as `...` too. The model config of the LoLCATs control sets the window to 128 and the feature dimension to 128, although its name says 64.

The main settings of the model config of `window_rope`:

```
attention:
  attention_type: lolcats_llama_lizard_v2
  feature_dim: 128
  window_size: 128
  num_meta: 4
  gla_norm: row
  alpha_init: 1.0
  window_rope: true
  softmax_attentions: []
```

APPENDIX D

All Runs and Scores

Table D.1 lists every training run of this thesis with its loss and its scores. The loss is the stage 1 validation loss for stage 1 models and the stage 2 validation loss for stage 2 models. The two losses have different scales. "A" is the share of the answer "A" on the MMLU subset. A dash means that the notes have no value.

Table D.1: All runs and scores. Accuracy in %.

Run	Stage	Loss	PIQA	ARC-Easy	MMLU subset	"A"
Teacher	–	–	74.4	65.3	33.7	–
Run 1, feature dimension 128	1	3.2549	57.6	39.0	22.5	95.4%
First Lizard model (Run 2)	2	2.252	67.95	54.8	24.9	98.6%
Feature dimension 32	1	3.4219	–	–	23.2	–
Paper LR, bf16	1	8.1641	55.8	34.1	25.3	21.8%
Paper LR, float32	1	4.9478	57.7	35.7	24.6	–
Config 1	1	3.9764	57.3	36.5	26.7	51.2%
Config 2	1	3.5092	57.5	35.6	23.2	–
Stage 2 on config 1	2	1.9158	67.7	54.9	23.5	77.5%
C1	1	3.8092	57.5	34.6	23.9	64.9%
R1	1	stopped	–	–	–	–
R1b	1	3.0542	55.9	29.9	24.9	90.2%
`window_rope`	1	1.2290	61.5	43.3	24.6	40.0%
`window_rope`	2	1.235	73.1	63.8	25.6	41.4%
`window_rope` + `hybrid`	1	1.6172	58.9	36.0	25.3	26.3%
LoLCATs control	1	0.4442	73.5	62.8	26.0	19.3%

On all MMLU questions, the teacher gets 31.7, the first Lizard model 23.3 and `window_rope` after stage 2 27.2. On ARC-Challenge, `window_rope` after stage 2 gets 36.1 (`acc_norm`).

APPENDIX E

MMLU for Each Subject

Table E.1 gives the 5-shot MMLU accuracy of each subject for the teacher, the first Lizard model and `window_rope` after stage 2. The values of the teacher and of `window_rope` come from the answer files of the full MMLU run. The values of the first Lizard model come from document 7 of the project notes. The last column is the difference between `window_rope` and the teacher.

Table E.1: MMLU accuracy for each subject, in %. n is the number of questions.

Subject	n	Teacher	First Lizard	`window_rope`	Difference
abstract algebra	100	27.0	22.0	25.0	−2.0
anatomy	135	38.5	21.5	31.9	−6.6
astronomy	152	24.3	17.8	24.3	+0.0
business ethics	100	36.0	30.0	24.0	−12.0
clinical knowledge	265	32.8	21.1	24.5	−8.3
college biology	144	26.4	25.0	26.4	+0.0
college chemistry	100	21.0	20.0	22.0	+1.0
college computer science	100	29.0	25.0	24.0	−5.0
college mathematics	100	24.0	22.0	20.0	−4.0
college medicine	173	28.3	20.8	27.7	−0.6
college physics	102	22.5	21.6	25.5	+3.0
computer security	100	49.0	32.0	30.0	−19.0
conceptual physics	235	33.6	26.4	34.0	+0.4
econometrics	114	26.3	23.7	20.2	−6.1
electrical engineering	145	26.2	24.1	24.1	−2.1
elementary mathematics	378	23.0	20.9	24.6	+1.6
formal logic	126	16.7	29.4	26.2	+9.5
global facts	100	21.0	18.0	24.0	+3.0
high school biology	310	29.0	18.4	23.9	−5.1
high school chemistry	203	23.6	18.7	25.6	+2.0
high school computer science	100	33.0	24.0	26.0	−7.0
high school european history	165	38.8	21.8	21.2	−17.6
high school geography	198	37.4	17.2	24.7	−12.7

Subject	n	Teacher	First Lizard	`window_rope`	Difference
high school government and politics	193	33.2	19.7	22.8	−10.4
high school macroeconomics	390	25.6	20.3	23.8	−1.8
high school mathematics	270	28.1	21.1	24.4	−3.7
high school microeconomics	238	25.6	21.0	26.9	+1.3
high school physics	151	22.5	19.2	26.5	+4.0
high school psychology	545	33.2	19.3	28.1	−5.1
high school statistics	216	31.0	15.3	17.1	−13.9
high school us history	204	30.4	25.0	24.5	−5.9
high school world history	237	31.6	27.0	28.3	−3.3
human aging	223	39.0	30.5	37.7	−1.3
human sexuality	131	35.1	26.0	30.5	−4.6
international law	121	43.0	24.0	46.3	+3.3
jurisprudence	108	37.0	25.9	35.2	−1.8
logical fallacies	163	27.0	23.9	22.1	−4.9
machine learning	112	34.8	31.2	30.4	−4.4
management	103	34.0	17.5	23.3	−10.7
marketing	234	42.3	28.6	32.5	−9.8
medical genetics	100	39.0	31.0	32.0	−7.0
miscellaneous	783	43.0	23.5	31.7	−11.3
moral disputes	346	28.3	24.9	27.2	−1.1
moral scenarios	895	23.8	23.8	23.9	+0.1
nutrition	306	38.2	22.2	30.1	−8.1
philosophy	311	32.8	18.6	29.6	−3.2
prehistory	324	38.0	21.6	28.4	−9.6
professional accounting	282	27.0	23.4	25.9	−1.1
professional law	1,534	25.2	24.6	24.0	−1.2
professional medicine	272	28.3	18.4	19.9	−8.4
professional psychology	612	31.2	25.0	31.5	+0.3
public relations	110	30.0	21.8	34.5	+4.5
security studies	245	37.1	18.8	23.7	−13.4
sociology	201	34.3	24.4	25.4	−8.9
us foreign policy	100	49.0	27.0	39.0	−10.0
virology	166	38.6	27.7	33.1	−5.5

Subject	n	Teacher	First Lizard	`window_rope`	Difference
world religions	171	43.3	32.2	31.0	−12.3
Mean over the subjects	14,042	31.7	23.3	27.2	−4.5

APPENDIX F

How to Reproduce the Results

The code is in the repository `mahdikhashan/lolcats` [16]. The commands below come from the project notes. They need a Hugging Face token with access to Llama-3.2-1B and to the checkpoint repository.

F.1 Training on Hugging Face Jobs

A full run of both stages with the original Lizard attention:

```
make hf-job IMAGE=mahdikhashan/lolcats HF_REPO=<hub repo> \
  HF_FLAVOR=h200 HF_TIMEOUT=6h
```

Stage 1 of `window_rope` alone, then stage 2 from its checkpoint:

```
make hf-job IMAGE=mahdikhashan/lolcats HF_REPO=<hub repo> \
  HF_FLAVOR=h200 HF_TIMEOUT=2h \
  ARGS="--model_config \
  distill_llama3_2_1b_lizard_v2_w128_fd128_m4_windowrope --no_finetune"
make hf-job-finetune IMAGE=mahdikhashan/lolcats HF_REPO=<hub repo> \
  DISTILL_CKPT=<path of the stage 1 checkpoint in the hub repo> \
  ARGS="--model_config \
  distill_llama3_2_1b_lizard_v2_w128_fd128_m4_windowrope"
```

The default `DISTILL_CKPT` of the Makefile is the checkpoint path of Run 1. Thus a stage 2 of another model needs the path of its own stage 1 checkpoint. The project notes do not record the exact command of the `window_rope` stage 2, so this command is a reconstruction from the Makefile. The other runs use the configs of Appendix C in `ARGS`.

F.2 Evaluation on a GPU machine with conda

```
CONDA_OVERRIDE_CUDA=12.4 conda env create -f environment.yaml
conda activate lolcats-env
export HF_TOKEN=...
scripts/compare_stages.sh            # teacher, stage 1, stage 2
MODELS=stage2 TASKS="piqa arc_easy" scripts/compare_stages.sh
MODELS="teacher stage2" TASKS=mmlu scripts/compare_stages.sh
MODELS=stage2 TASKS="arc_challenge hellaswag winogrande" \
  scripts/compare_stages.sh
```

F.3 XAI scripts

```
python scripts/layer_mse.py compute results/layer_mse/<name>.json \
  --model_config <model config> --distill_config <distill config>
python scripts/attention_weights.py compute \
  results/attention_weights/<name>.json \
  --model_config <model config> --distill_config <distill config>
python scripts/position_mse.py compute \
  results/position_mse/<name>.json \
  --from_json docs/experiments/xai-layer-wise-mse/<name>.json
python scripts/branch_fit.py compute \
  results/branch_fit/<name>.json \
  --from_json docs/experiments/xai-layer-wise-mse/<name>.json
```

F.4 This document

The source of this thesis is in `docs/thesis/src`. The command `make figures` draws the new plots with matplotlib. The command `make` builds the PDF with `latexmk`. The command `make lint` checks the text with the ASD-STE100 linter of the repository.

APPENDIX G

Use of AI Tools

G.1 Claude Code

Claude Code, an AI coding assistant, did most of the implementation and documentation work as pull requests. The author gave each task in a chat, read the pull requests and merged them. The repository `jku-thesis` has 4 such pull requests, and the repository `lolcats` has more than 90. Examples are the port of Lizard to LoLCATs, the Docker image, the Hugging Face Jobs targets and the NaN fix. Further examples are the evaluation scripts, Lizard attention v2, the XAI scripts and all project notes. The author ran the training jobs and the evaluations and gave the results back as logs and files.

The project notes follow a controlled language (ASD-STE100 rules), and a linter checks each document. The notes mark measured numbers and estimated numbers. They give the source of each number.

G.2 ChatGPT

Some project notes keep answers from ChatGPT without changes. These are the definitions appendix, the evaluation metrics, three answers about reproduction studies and an answer about XAI methods for MMLU. Separate notes review these answers against the measured results. This thesis does not use them as sources of facts. Appendix A of this thesis gives new definitions.

G.3 This thesis

Claude Code wrote the first draft of this thesis from the project notes and the outline in `docs/thesis/outline.md`. It also drew the new plots from the numbers in the notes. It computed the table of Appendix E from the answer files of the full MMLU run. The author must check every statement, number and reference. Some references come from web searches, and their full texts were not read. The bibliography marks them.

Bibliography

[1] Simran Arora et al. Simple linear attention language models balance the recall-throughput tradeoff. arXiv:2402.18668, 2024.

[2] Nora Belrose et al. Eliciting latent predictions from transformers with the tuned lens. arXiv:2303.08112, 2023.

[3] Aviv Bick, Eric Xing, and Albert Gu. Understanding the skill gap in recurrent language models: The role of the gather-and-aggregate mechanism. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2504.18574.

[4] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *AAAI Conference on Artificial Intelligence*, 2020.

[5] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. arXiv:1803.05457, 2018.

[6] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

[7] Abhimanyu Dubey et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.

[8] Leo Gao et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2021. This thesis uses commit b281b09.

[9] Daniel Goldstein et al. RADLADS: Rapid attention distillation to linear attention decoders at scale. arXiv:2505.03005, 2025.

[10] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. arXiv:2404.15255, 2024.

[11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

[12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[14] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.

[15] Mahdi Khashan. jku-thesis: reference implementation of Lizard attention. `https://github.com/mahdikhashan/jku-thesis`, 2026.

[16] Mahdi Khashan. lolcats: Lizard attention in the LoLCATs code base. `https://github.com/mahdikhashan/lolcats`, 2026. Fork of HazyResearch/lolcats. Project notes in docs/.

[17] KL. KL-guided layer selection for hybrid linear attention. arXiv:2512.20569, 2025. ICLR 2026. Found by a web search. The full text was not read.

[18] Disen Lan, Weigao Sun, Jiaxi Hu, Jusen Du, and Yu Cheng. Liger: Linearizing large language models to gated recurrent structures. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.01496.

[19] LoLA. LoLA: Low-rank linear attention with sparse caching. arXiv:2505.23666, 2025. Found by a web search. The full text was not read.

[20] Chien Van Nguyen et al. Lizard: An efficient linearization framework for large language models. arXiv:2507.09025, version 4, 2025. Also in the proceedings of ACL 2026.

[21] Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. Transformer Circuits Thread, 2022.

[22] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *AAAI Conference on Artificial Intelligence*, 2020.

[23] STILL. STILL. arXiv:2602.02180, 2026. Found by a web search. The full text was not read.

[24] Stuck. Stuck on "A". arXiv:2608.02689, 2026. Found by a web search. The full text was not read.

[25] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. arXiv:2104.09864, 2021.

[26] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023. This thesis uses the cleaned version yahma/alpaca-cleaned on Hugging Face.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[28] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[29] Guangxuan Xiao et al. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10819.

[30] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning (ICML)*, 2024.

[31] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[32] Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2402.04347.

[33] Michael Zhang, Simran Arora, Rahul Chalamala, Alan Wu, Benjamin Spector, Aaryan Singhal, Krithik Ramesh, and Christopher Ré. LoLCATs: On low-rank linearizing of large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10254.